

Telco Customer Churn

```
In [ ]: import pandas as pd
from sklearn.preprocessing import LabelEncoder, PowerTransformer, OrdinalEncoder, C
import seaborn as sns
from sklearn.model_selection import train_test_split
from matplotlib import pyplot as plt
import numpy as np
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from imblearn.pipeline import Pipeline as IMBPipeline
from imblearn.over_sampling import SMOTE
from sklearn.impute import SimpleImputer
from sklearn.metrics import precision_score, recall_score, confusion_matrix, Con
from sklearn.model_selection import RandomizedSearchCV, GridSearchCV
from sklearn.ensemble import GradientBoostingClassifier
from xgboost import XGBClassifier
import time
```

```
In [ ]: df = pd.read_csv('https://raw.githubusercontent.com/KarthiObli/TelcoCustomerChur
```

```
In [ ]: df.head()
```

```
Out[ ]:   customerID  gender  SeniorCitizen  Partner  Dependents  tenure  PhoneService  Mul
```

0	7590-VHVEG	Female	0	Yes	No	1	No
1	5575-GNVDE	Male	0	No	No	34	Yes
2	3668-QPYBK	Male	0	No	No	2	Yes
3	7795-CFOCW	Male	0	No	No	45	No
4	9237-HQITU	Female	0	No	No	2	Yes

5 rows × 21 columns



```
In [ ]: df.drop('customerID', axis=1, inplace=True)
```

```
In [ ]: df_corr = df[:, :]
```

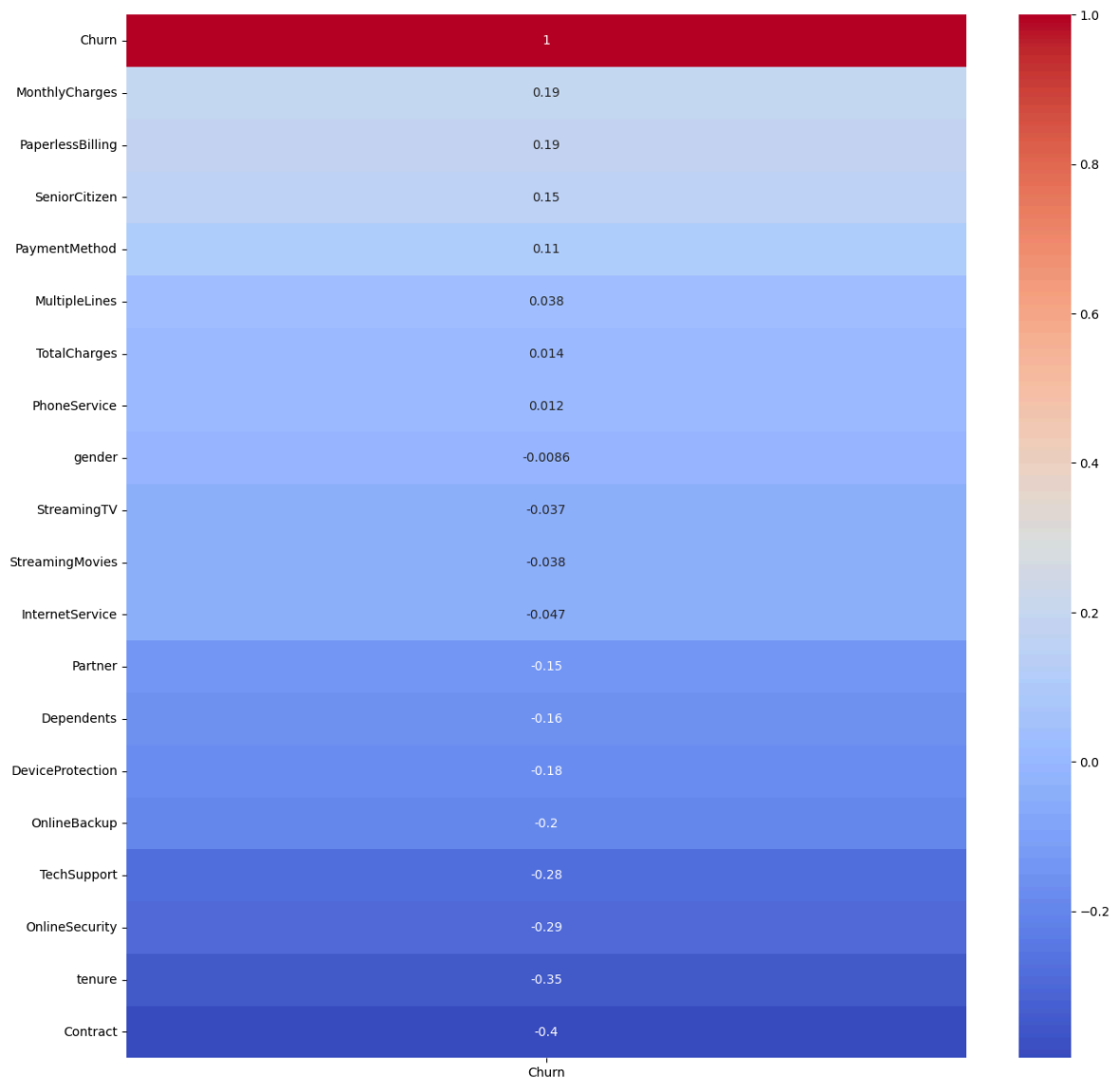
```
In [ ]: le = LabelEncoder()
```

```
In [ ]: for i in df_corr.select_dtypes(include='object').columns:  
        df_corr[i] = le.fit_transform(df_corr[i])
```

```
In [ ]: corr = df_corr.corr()
```

```
In [ ]: plt.figure(figsize=(15,15))  
sns.heatmap(corr[['Churn']].sort_values(by='Churn',ascending=False),annot=True,c
```

```
Out[ ]: <Axes: >
```



```
In [ ]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   gender                 7043 non-null   object
1   SeniorCitizen          7043 non-null   int64
2   Partner                7043 non-null   object
3   Dependents             7043 non-null   object
4   tenure                 7043 non-null   int64
5   PhoneService           7043 non-null   object
6   MultipleLines          7043 non-null   object
7   InternetService        7043 non-null   object
8   OnlineSecurity         7043 non-null   object
9   OnlineBackup           7043 non-null   object
10  DeviceProtection       7043 non-null   object
11  TechSupport            7043 non-null   object
12  StreamingTV            7043 non-null   object
13  StreamingMovies        7043 non-null   object
14  Contract               7043 non-null   object
15  PaperlessBilling       7043 non-null   object
16  PaymentMethod          7043 non-null   object
17  MonthlyCharges         7043 non-null   float64
18  TotalCharges           7043 non-null   object
19  Churn                  7043 non-null   object
dtypes: float64(1), int64(2), object(17)
memory usage: 1.1+ MB

```

```
In [ ]: df['TotalCharges'] = pd.to_numeric(df['TotalCharges'].str.strip(),errors='coerce')
```

```
In [ ]: df['TotalCharges'] = df['TotalCharges'].astype('float')
```

```
In [ ]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   gender                 7043 non-null  object
1   SeniorCitizen          7043 non-null  int64
2   Partner                7043 non-null  object
3   Dependents             7043 non-null  object
4   tenure                 7043 non-null  int64
5   PhoneService           7043 non-null  object
6   MultipleLines           7043 non-null  object
7   InternetService        7043 non-null  object
8   OnlineSecurity         7043 non-null  object
9   OnlineBackup           7043 non-null  object
10  DeviceProtection       7043 non-null  object
11  TechSupport            7043 non-null  object
12  StreamingTV            7043 non-null  object
13  StreamingMovies        7043 non-null  object
14  Contract               7043 non-null  object
15  PaperlessBilling       7043 non-null  object
16  PaymentMethod          7043 non-null  object
17  MonthlyCharges         7043 non-null  float64
18  TotalCharges           7032 non-null  float64
19  Churn                  7043 non-null  object
dtypes: float64(2), int64(2), object(16)
memory usage: 1.1+ MB

```

```
In [ ]: df.describe()
```

```
Out[ ]:
```

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges
count	7043.000000	7043.000000	7043.000000	7032.000000
mean	0.162147	32.371149	64.761692	2283.300441
std	0.368612	24.559481	30.090047	2266.771362
min	0.000000	0.000000	18.250000	18.800000
25%	0.000000	9.000000	35.500000	401.450000
50%	0.000000	29.000000	70.350000	1397.475000
75%	0.000000	55.000000	89.850000	3794.737500
max	1.000000	72.000000	118.750000	8684.800000

```
In [ ]: df['SeniorCitizen'] = np.where(df['SeniorCitizen']==1, 'Yes', 'No')
```

Train Test Split

```
In [ ]: x = df.drop('Churn',axis=1)
```

```
In [ ]: y = df[['Churn']]
```

```
In [ ]: x_train, x_test, y_train, y_test = train_test_split(x,y,random_state=21,test_siz
```

```
In [ ]: x_train.reset_index(drop=True,inplace=True)
x_test.reset_index(drop=True,inplace=True)
y_train.reset_index(drop=True,inplace=True)
y_test.reset_index(drop=True,inplace=True)
```

Null Handling

```
In [ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   gender                7043 non-null   object
1   SeniorCitizen         7043 non-null   object
2   Partner               7043 non-null   object
3   Dependents            7043 non-null   object
4   tenure                7043 non-null   int64
5   PhoneService          7043 non-null   object
6   MultipleLines         7043 non-null   object
7   InternetService       7043 non-null   object
8   OnlineSecurity        7043 non-null   object
9   OnlineBackup          7043 non-null   object
10  DeviceProtection      7043 non-null   object
11  TechSupport           7043 non-null   object
12  StreamingTV           7043 non-null   object
13  StreamingMovies       7043 non-null   object
14  Contract              7043 non-null   object
15  PaperlessBilling      7043 non-null   object
16  PaymentMethod         7043 non-null   object
17  MonthlyCharges        7043 non-null   float64
18  TotalCharges          7032 non-null   float64
19  Churn                 7043 non-null   object
dtypes: float64(2), int64(1), object(17)
memory usage: 1.1+ MB
```

```
In [ ]: Impute_Total_Charges = SimpleImputer(strategy = 'mean')
x_train['TotalCharges'] = Impute_Total_Charges.fit_transform(x_train[['TotalCharges']])
x_test['TotalCharges'] = Impute_Total_Charges.fit_transform(x_test[['TotalCharges']])
```

Outlier Handling

```
In [ ]: x_train.head()
```

Out[]: **gender** **SeniorCitizen** **Partner** **Dependents** **tenure** **PhoneService** **MultipleLines** **In**

0	Male	No	No	No	2	Yes	Yes
----------	------	----	----	----	---	-----	-----

1	Female	No	No	No	34	Yes	Yes
----------	--------	----	----	----	----	-----	-----

2	Female	No	No	No	9	Yes	No
----------	--------	----	----	----	---	-----	----

3	Male	No	Yes	No	67	Yes	Yes
----------	------	----	-----	----	----	-----	-----

4	Male	No	No	No	71	No	No phone service
----------	------	----	----	----	----	----	------------------

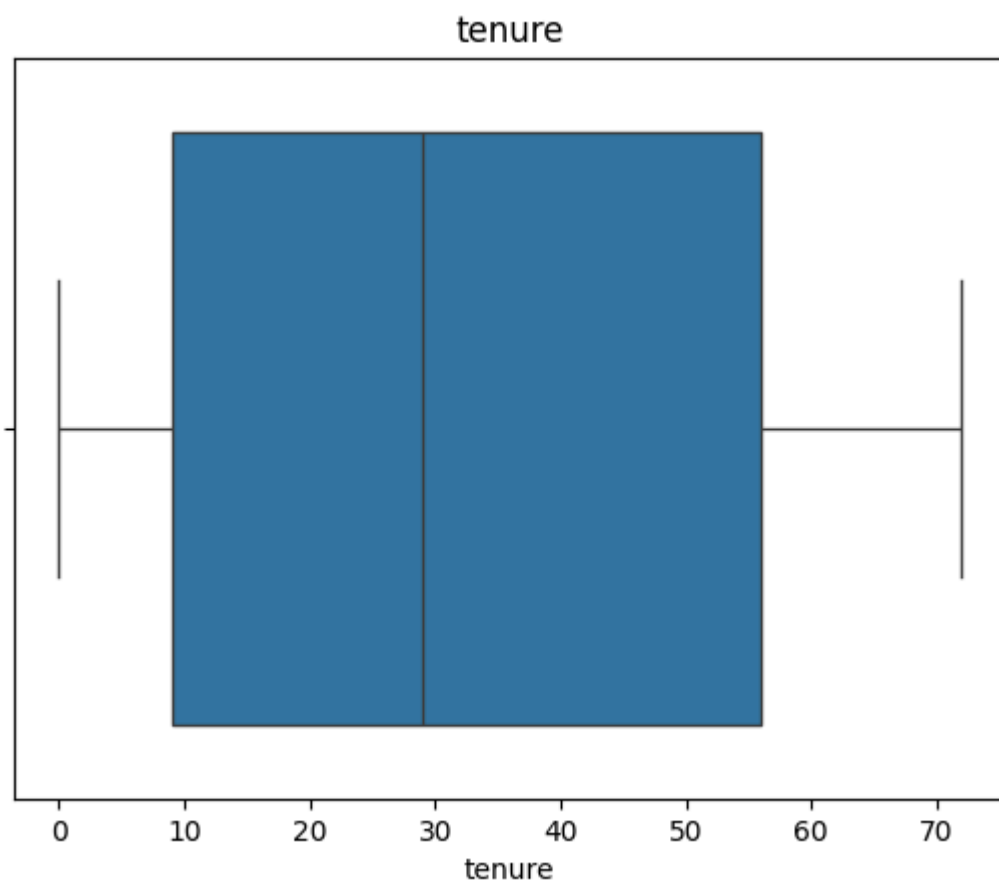


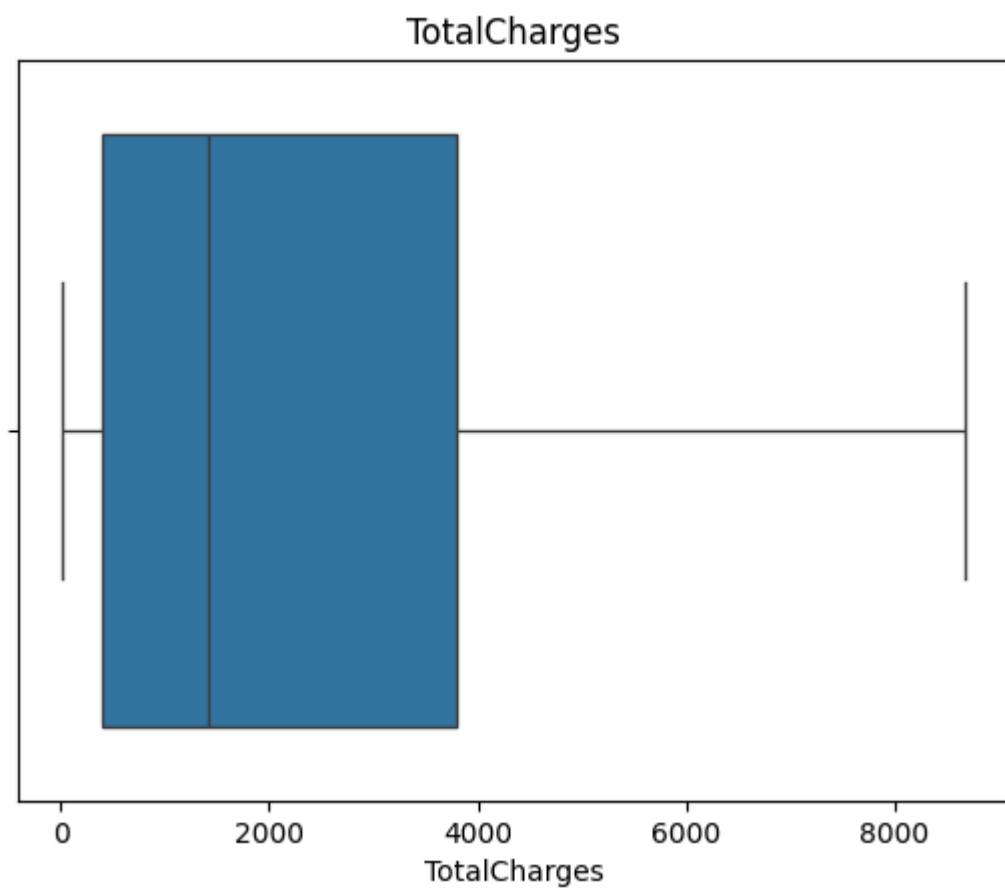
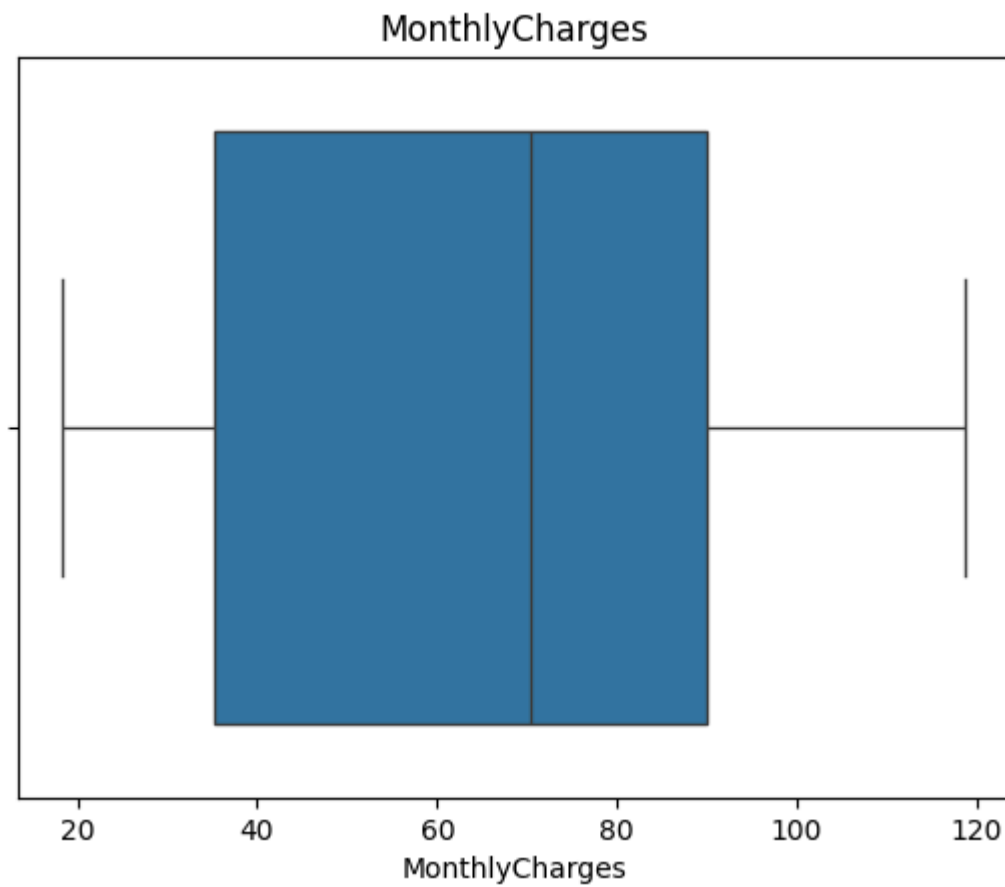
```
In [ ]: numerical_columns = [i for i in x_train.select_dtypes(['int', 'float']).columns]
```

```
In [ ]: numerical_columns
```

```
Out[ ]: ['tenure', 'MonthlyCharges', 'TotalCharges']
```

```
In [ ]: for i in numerical_columns:
sns.boxplot(x=x_train[i])
plt.title(i)
plt.show()
```





Outlier handling is not required

Skew Data handling

```
In [ ]: for i in numerical_columns:
        sns.distplot(x=x_train[i])
        plt.title(i)
        plt.show()
```

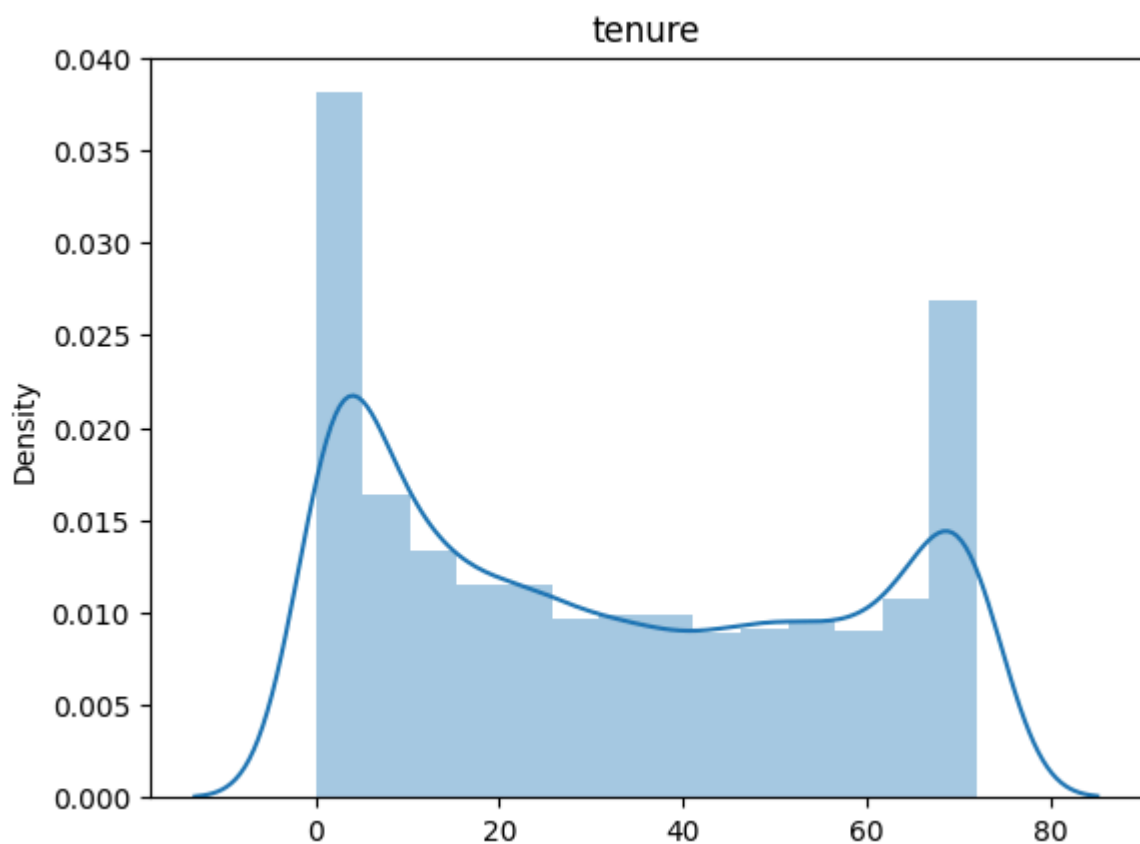
/tmp/ipykernel_2917/1145699575.py:2: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(x=x_train[i])
```



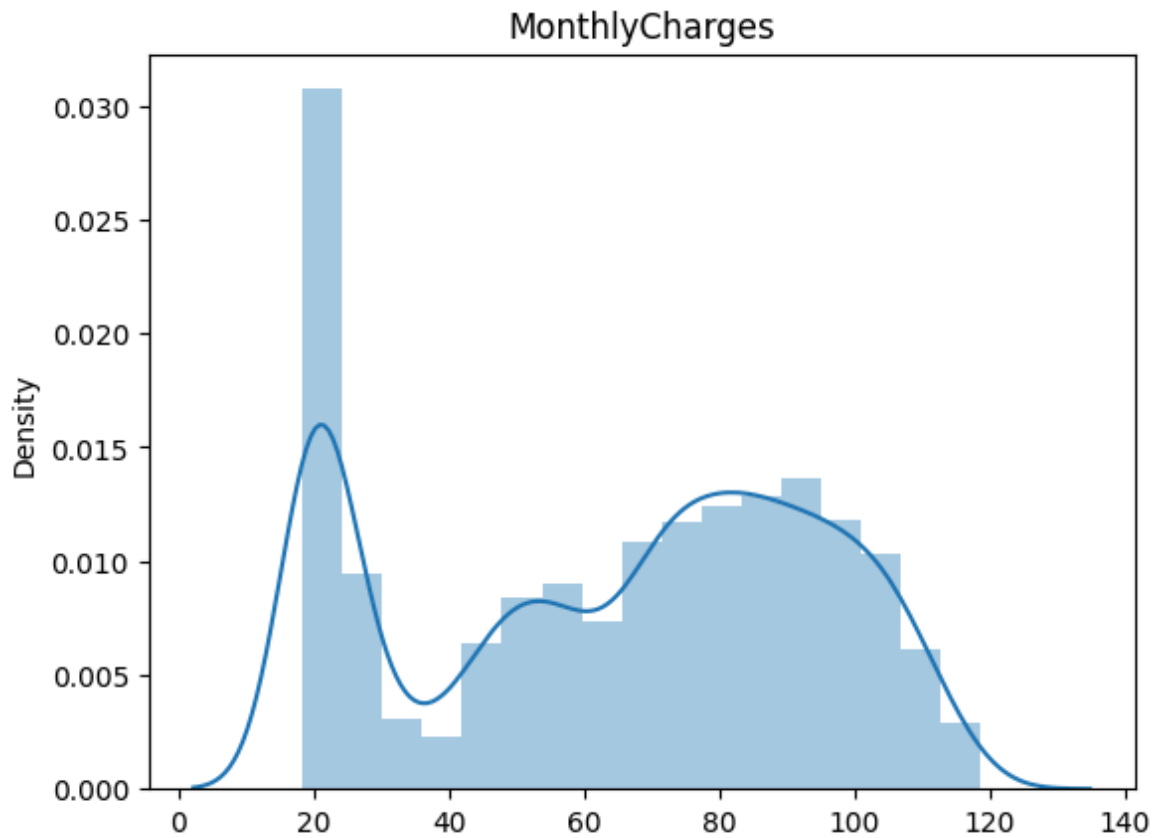
/tmp/ipykernel_2917/1145699575.py:2: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(x=x_train[i])
```

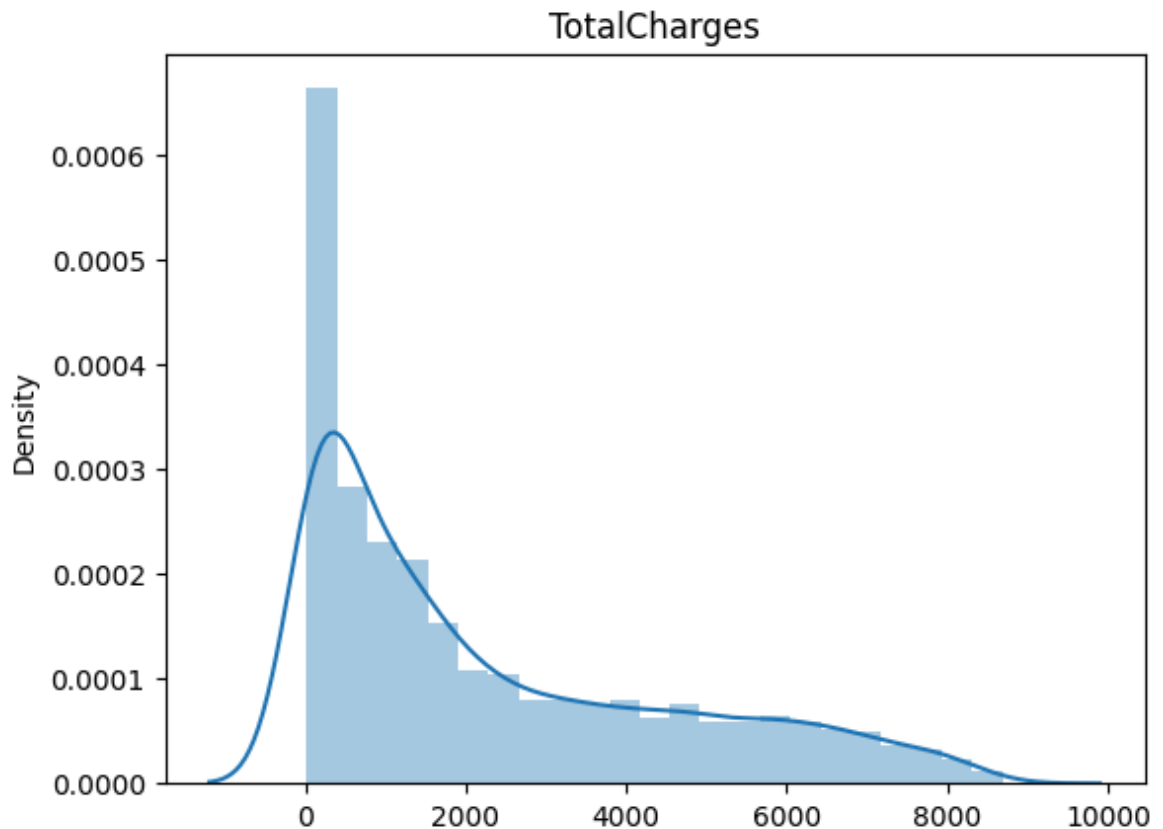
/tmp/ipykernel_2917/1145699575.py:2: UserWarning:

``distplot`` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either ``displot`` (a figure-level function with similar flexibility) or ``histplot`` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(x=x_train[i])
```



```
In [ ]: for i in numerical_columns:
        print(f'Skew of {i} is {x_train[i].skew()}')
```

Skew of tenure is 0.24154307949177942
 Skew of MonthlyCharges is -0.2180067984814727
 Skew of TotalCharges is 0.959567310717357

```
In [ ]: PT = PowerTransformer(method='yeo-johnson')
```

```
In [ ]: x_train['TotalCharges'] = PT.fit_transform(x_train[['TotalCharges']])
```

```
In [ ]: x_test['TotalCharges'] = PT.transform(x_test[['TotalCharges']])
```

```
In [ ]: for i in numerical_columns:
        sns.distplot(x=x_train[i])
        plt.title(i)
        plt.show()
```

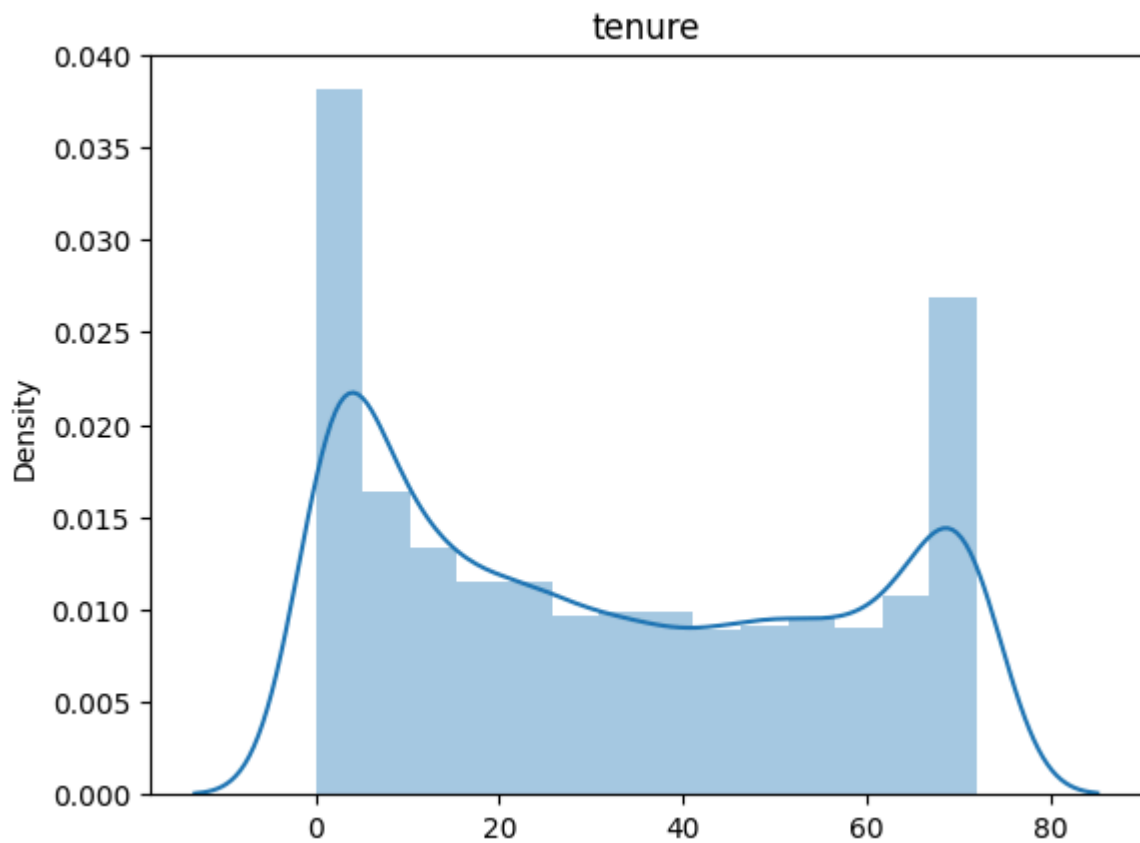
/tmp/ipykernel_2917/1145699575.py:2: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(x=x_train[i])
```



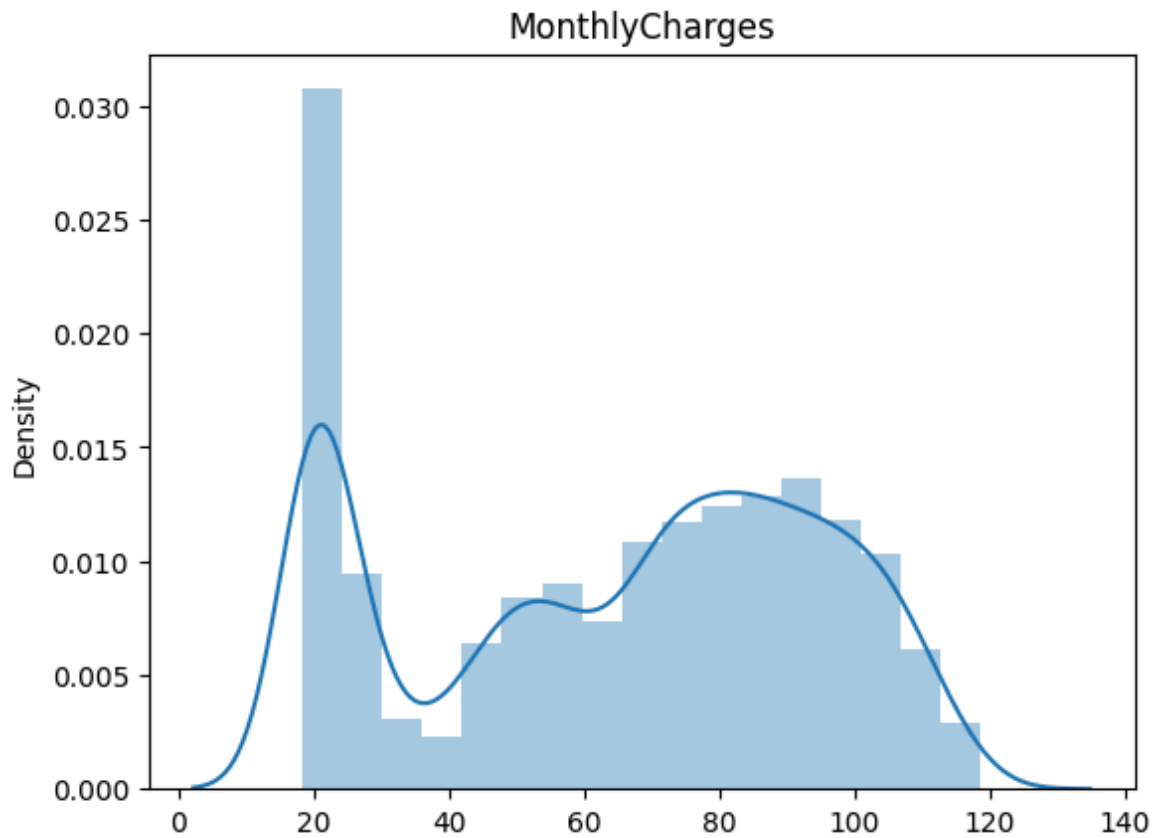
/tmp/ipykernel_2917/1145699575.py:2: UserWarning:

``distplot`` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either ``displot`` (a figure-level function with similar flexibility) or ``histplot`` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(x=x_train[i])
```



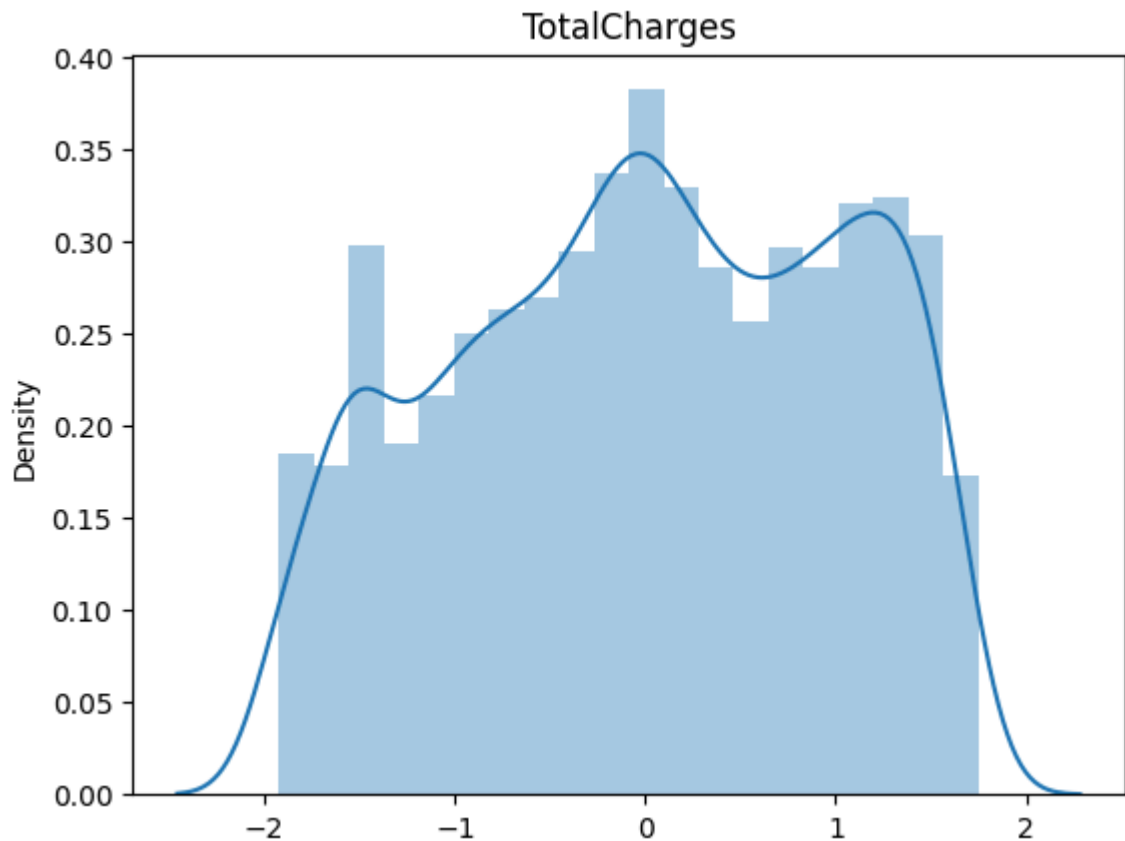
/tmp/ipykernel_2917/1145699575.py:2: UserWarning:

``distplot`` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either ``displot`` (a figure-level function with similar flexibility) or ``histplot`` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(x=x_train[i])
```



```
In [ ]: for i in numerical_columns:
        print(f'Skew of {i} is {x_train[i].skew()}')
```

Skew of tenure is 0.24154307949177942
 Skew of MonthlyCharges is -0.2180067984814727
 Skew of TotalCharges is -0.1453161207648667

```
In [ ]: x_train.head()
```

```
Out [ ]:   gender  SeniorCitizen  Partner  Dependents  tenure  PhoneService  MultipleLines  In
```

```
0    Male             No      No           No         2             Yes             Yes
```

```
1    Female           No      No           No        34             Yes             Yes
```

```
2    Female           No      No           No         9             Yes             No
```

```
3    Male             No      Yes          No        67             Yes             Yes
```

```
4    Male             No      No           No        71             No           No phone
                                     service
```



```
In [ ]: categorical_columns = [i for i in x_train.select_dtypes('object').columns]
```

Encoding and Feature Scaling

```
In [ ]: x_train.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 5634 entries, 0 to 5633  
Data columns (total 19 columns):  
#   Column                Non-Null Count  Dtype    
---  ---                  
0   gender                5634 non-null  object   
1   SeniorCitizen         5634 non-null  object   
2   Partner               5634 non-null  object   
3   Dependents            5634 non-null  object   
4   tenure                5634 non-null  int64    
5   PhoneService          5634 non-null  object   
6   MultipleLines         5634 non-null  object   
7   InternetService       5634 non-null  object   
8   OnlineSecurity        5634 non-null  object   
9   OnlineBackup          5634 non-null  object   
10  DeviceProtection      5634 non-null  object   
11  TechSupport           5634 non-null  object   
12  StreamingTV           5634 non-null  object   
13  StreamingMovies       5634 non-null  object   
14  Contract              5634 non-null  object   
15  PaperlessBilling      5634 non-null  object   
16  PaymentMethod         5634 non-null  object   
17  MonthlyCharges        5634 non-null  float64  
18  TotalCharges          5634 non-null  float64  
dtypes: float64(2), int64(1), object(16)  
memory usage: 836.4+ KB
```

```
In [ ]: x_train['gender'].unique()
```

```
Out[ ]: array(['Male', 'Female'], dtype=object)
```

```
In [ ]: x_train['SeniorCitizen'].unique()
```

```
Out[ ]: array(['No', 'Yes'], dtype=object)
```

```
In [ ]: x_train['Partner'].unique()
```

```
Out[ ]: array(['No', 'Yes'], dtype=object)
```

```
In [ ]: x_train['Dependents'].unique()
```

```
Out[ ]: array(['No', 'Yes'], dtype=object)
```

```
In [ ]: x_train['PhoneService'].unique()
```

```
Out[ ]: array(['Yes', 'No'], dtype=object)
```

```
In [ ]: x_train['MultipleLines'].unique()
```

```
Out[ ]: array(['Yes', 'No', 'No phone service'], dtype=object)
```

```
In [ ]: x_train['InternetService'].unique()
```

```
Out[ ]: array(['Fiber optic', 'DSL', 'No'], dtype=object)
```

```

In [ ]: x_train['OnlineSecurity'].unique()

Out[ ]: array(['No', 'Yes', 'No internet service'], dtype=object)

In [ ]: x_train['OnlineBackup'].unique()

Out[ ]: array(['Yes', 'No internet service', 'No'], dtype=object)

In [ ]: x_train['DeviceProtection'].unique()

Out[ ]: array(['No', 'Yes', 'No internet service'], dtype=object)

In [ ]: x_train['TechSupport'].unique()

Out[ ]: array(['No', 'Yes', 'No internet service'], dtype=object)

In [ ]: x_train['StreamingMovies'].unique()

Out[ ]: array(['No', 'Yes', 'No internet service'], dtype=object)

In [ ]: x_train['Contract'].unique()

Out[ ]: array(['Month-to-month', 'Two year', 'One year'], dtype=object)

In [ ]: x_train['PaperlessBilling'].unique()

Out[ ]: array(['Yes', 'No'], dtype=object)

In [ ]: x_train['PaymentMethod'].unique()

Out[ ]: array(['Electronic check', 'Mailed check', 'Bank transfer (automatic)',
              'Credit card (automatic)'], dtype=object)

In [ ]: x_train['StreamingTV'].unique()

Out[ ]: array(['No', 'Yes', 'No internet service'], dtype=object)

In [ ]: ColumnTransformer_Preprocessor = ColumnTransformer([('Ordinal_encoding', OrdinalEncoder)])

In [ ]: ColumnTransformer_Preprocessor_y = ColumnTransformer([('Encoding', OrdinalEncoder)])

In [ ]: pipelines = Pipeline([('Preprocessing', ColumnTransformer_Preprocessor)])

In [ ]: pipelines_y = Pipeline([('Preprocessing_y', ColumnTransformer_Preprocessor_y)])

In [ ]: x_train = pd.DataFrame(pipelines.fit_transform(x_train), columns=pipelines.named_steps['Preprocessing'].get_feature_names_out())

In [ ]: x_test = pd.DataFrame(pipelines.transform(x_test), columns=pipelines.named_steps['Preprocessing'].get_feature_names_out())

In [ ]: y_train = pd.DataFrame(pipelines_y.fit_transform(y_train), columns=pipelines_y.named_steps['Encoding'].get_feature_names_out())

In [ ]: y_test = pd.DataFrame(pipelines_y.transform(y_test), columns=pipelines_y.named_steps['Encoding'].get_feature_names_out())

In [ ]: x_train.head()

```

Out[]:

	Ordinal_encoding_gender	Ordinal_encoding_SeniorCitizen	Ordinal_encoding_Partner
0	1.0	0.0	0.0
1	0.0	0.0	0.0
2	0.0	0.0	0.0
3	1.0	0.0	1.0
4	1.0	0.0	0.0

5 rows × 40 columns



Hyper parameter tuning

Random Forest Classifier

```
In [ ]: RF_pipeline = IMBPipeline([('smote', SMOTE(random_state=42)), ('model', RandomForest
```

```
In [ ]: params = {'model__n_estimators': [100, 150, 200, 250], 'model__criterion': ['gini', '
```

```
In [ ]: RandomSearch_RF = RandomizedSearchCV(estimator = RF_pipeline, param_distribution
```

```
In [ ]: start_time = time.time()
RandomSearch_RF.fit(x_train, y_train.iloc[:, 0])
end_time = time.time()
print(f'Time Taken in mins : {(end_time - start_time) / 60}')
```

Time Taken in mins : 3.0619815389315286

```
In [ ]: RandomSearch_RF_Results = pd.DataFrame(RandomSearch_RF.cv_results_)
```

```
In [ ]: RandomSearch_RF_Results['Train_Score_Minus_Test_Score'] = RandomSearch_RF_Result
```

```
In [ ]: RandomSearch_RF_Results.columns
```

```
Out[ ]: Index(['mean_fit_time', 'std_fit_time', 'mean_score_time', 'std_score_time',
              'param_model__n_estimators', 'param_model__min_samples_split',
              'param_model__max_samples', 'param_model__max_features',
              'param_model__max_depth', 'param_model__criterion', 'params',
              'split0_test_score', 'split1_test_score', 'split2_test_score',
              'split3_test_score', 'split4_test_score', 'mean_test_score',
              'std_test_score', 'rank_test_score', 'split0_train_score',
              'split1_train_score', 'split2_train_score', 'split3_train_score',
              'split4_train_score', 'mean_train_score', 'std_train_score',
              'Train_Score_Minus_Test_Score'],
              dtype='object')
```

```
In [ ]: RandomSearch_RF_Results[['param_model__n_estimators', 'param_model__min_samples_
```


Out[]: **param_model_n_estimators** **param_model_min_samples_split** **param_model_max_sai**

3	250	12
11	100	10
1	200	10
18	200	11
0	150	11
4	250	12
21	200	10
6	250	10
23	200	10
24	200	12
20	150	11
7	200	10
9	150	12
22	200	11
15	250	10
19	200	10
2	100	12
13	250	12
14	250	11
12	100	12
5	100	12
17	100	12
10	250	11
8	250	11
16	250	11



In []: RandomSearch_RF.best_params_

```
Out[ ]: {'model__n_estimators': 250,  
         'model__min_samples_split': 12,  
         'model__max_samples': 0.4,  
         'model__max_features': 'log2',  
         'model__max_depth': 5,  
         'model__criterion': 'gini'}
```

Gradient Boosting Classifier

```
In [ ]: GBC_pipeline = IMBPipeline([('smote', SMOTE(random_state=42)), ('model', GradientBo
```

```
In [ ]: params = {'model__learning_rate' : [0.1, 0.2, 0.3], 'model__n_estimators' : [100
```

```
In [ ]: RandomSearch_GBC = RandomizedSearchCV(estimator = GBC_pipeline, param_distributi
```

```
In [ ]: start_time = time.time()
RandomSearch_GBC.fit(x_train, y_train.iloc[:,0])
end_time = time.time()
print(f'Time Taken in mins : {(end_time-start_time)/60}')
```

Time Taken in mins : 11.38887213865916

```
In [ ]: RandomSearch_GBC_Results = pd.DataFrame(RandomSearch_GBC.cv_results_)
```

```
In [ ]: RandomSearch_GBC_Results['Train_Score_Minus_Test_Score'] = RandomSearch_GBC_Resu
```

```
In [ ]: RandomSearch_GBC_Results.columns
```

```
Out[ ]: Index(['mean_fit_time', 'std_fit_time', 'mean_score_time', 'std_score_time',
              'param_model__n_estimators', 'param_model__min_samples_split',
              'param_model__min_samples_leaf', 'param_model__max_leaf_nodes',
              'param_model__max_depth', 'param_model__learning_rate', 'params',
              'split0_test_score', 'split1_test_score', 'split2_test_score',
              'split3_test_score', 'split4_test_score', 'mean_test_score',
              'std_test_score', 'rank_test_score', 'split0_train_score',
              'split1_train_score', 'split2_train_score', 'split3_train_score',
              'split4_train_score', 'mean_train_score', 'std_train_score',
              'Train_Score_Minus_Test_Score'],
              dtype='object')
```

```
In [ ]: RandomSearch_GBC_Results[['param_model__n_estimators', 'param_model__min_samples
```

Out[]: **param_model_n_estimators** **param_model_min_samples_split** **param_model_min_san**

19	100	20
5	125	20
4	150	20
22	100	15
20	150	10
8	125	15
0	150	15
3	125	10
9	150	10
13	125	10
1	125	15
15	100	10
2	150	15
18	125	20
6	150	10
24	100	10
21	100	10
11	125	15
12	150	10
16	150	10
17	150	15
7	100	15
23	125	15
14	100	10
10	125	20



In []: RandomSearch_GBC.best_params_

```
Out[ ]: {'model__n_estimators': 100,  
         'model__min_samples_split': 20,  
         'model__min_samples_leaf': 8,  
         'model__max_leaf_nodes': 15,  
         'model__max_depth': 10,  
         'model__learning_rate': 0.1}
```

XGBoost

```
In [ ]: XGBC_pipeline = IMBPipeline([('smote', SMOTE(random_state=42)), ('model', XGBCClassi
```

```
In [ ]: params = {'model__n_estimators' : [50,100,150,200,250], 'model__learning_rate' :
```

```
In [ ]: RandomSearch_XGBC = RandomizedSearchCV(estimator = XGBC_pipeline, param_distribu
```

```
In [ ]: start_time = time.time()
RandomSearch_XGBC.fit(x_train,y_train)
end_time = time.time()
print(f'Time Taken in mins : {(end_time-start_time)/60}')
```

Time Taken in mins : 3.5248031417528787

```
In [ ]: RandomSearch_XGBC_Results = pd.DataFrame(RandomSearch_XGBC.cv_results_)
```

```
In [ ]: RandomSearch_XGBC_Results['Train_Score_Minus_Test_Score'] = RandomSearch_XGBC_Re
```

```
In [ ]: RandomSearch_XGBC_Results.columns
```

```
Out[ ]: Index(['mean_fit_time', 'std_fit_time', 'mean_score_time', 'std_score_time',
               'param_model__reg_lambda', 'param_model__reg_alpha',
               'param_model__n_estimators', 'param_model__max_depth',
               'param_model__learning_rate', 'params', 'split0_test_score',
               'split1_test_score', 'split2_test_score', 'split3_test_score',
               'split4_test_score', 'mean_test_score', 'std_test_score',
               'rank_test_score', 'split0_train_score', 'split1_train_score',
               'split2_train_score', 'split3_train_score', 'split4_train_score',
               'mean_train_score', 'std_train_score', 'Train_Score_Minus_Test_Score'],
              dtype='object')
```

```
In [ ]: RandomSearch_XGBC_Results[['param_model__reg_lambda', 'param_model__reg_alpha',
                                   'param_model__n_estimators', 'param_model__max_depth',
                                   'param_model__learning_rate', 'mean_train_score', 'mean_test_score', 'Train
```

Out[]:

	param_model_reg_lambda	param_model_reg_alpha	param_model_n_estimators	pa
2	2	0.5	50	
19	5	0.5	50	
17	3	0.5	50	
23	1	0.5	150	
22	5	0.1	150	
5	2	0.2	100	
4	2	0.2	50	
15	2	0.5	50	
3	3	0.4	50	
11	3	0.5	100	
1	4	0.1	250	
9	2	0.2	250	
13	3	0.4	50	
6	4	0.5	100	
0	1	0.4	50	
21	2	0.2	150	
8	1	0.4	100	
16	3	0.1	200	
12	1	0.5	200	
18	3	0.2	250	
24	2	0.1	100	
20	5	0.3	250	
7	3	0.4	200	
14	1	0.3	250	
10	1	0.4	150	

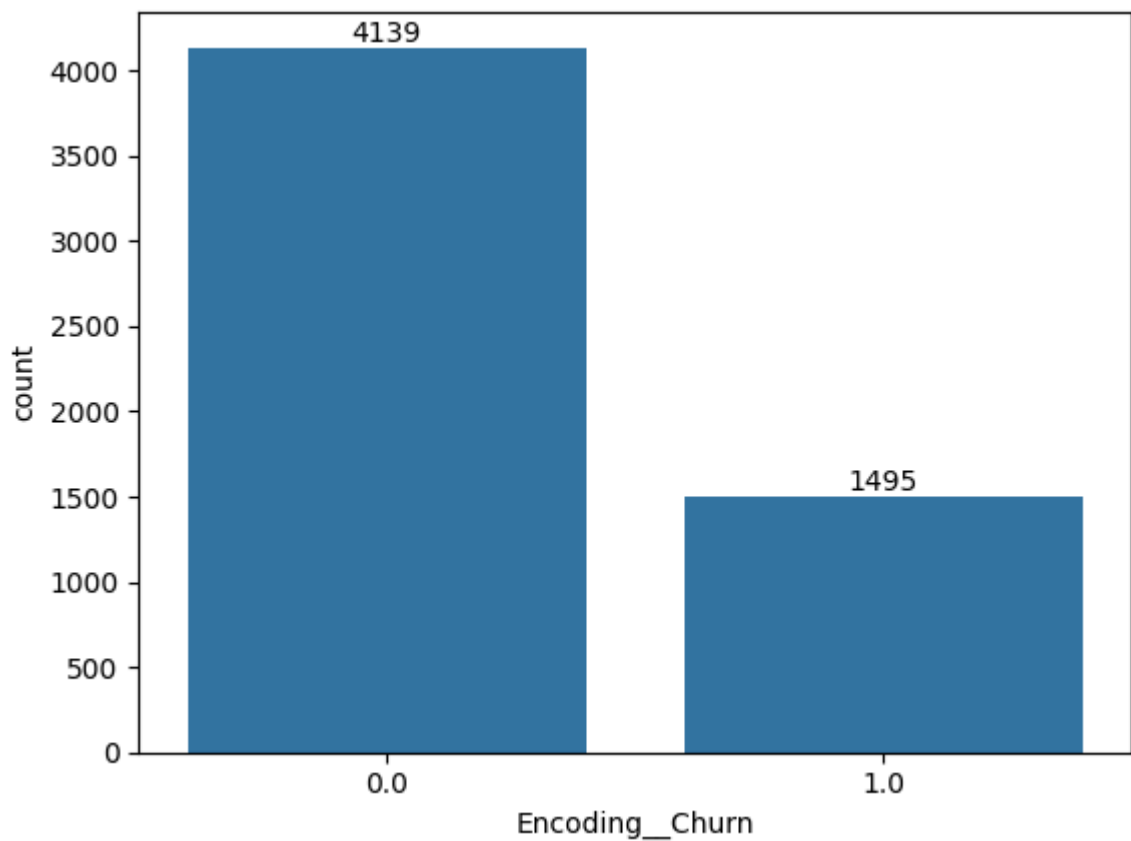


In []: RandomSearch_XGBC.best_params_

Out[]: {'model__reg_lambda': 2,
'model__reg_alpha': 0.5,
'model__n_estimators': 50,
'model__max_depth': 5,
'model__learning_rate': 0.1}

Imabalnced Data handling

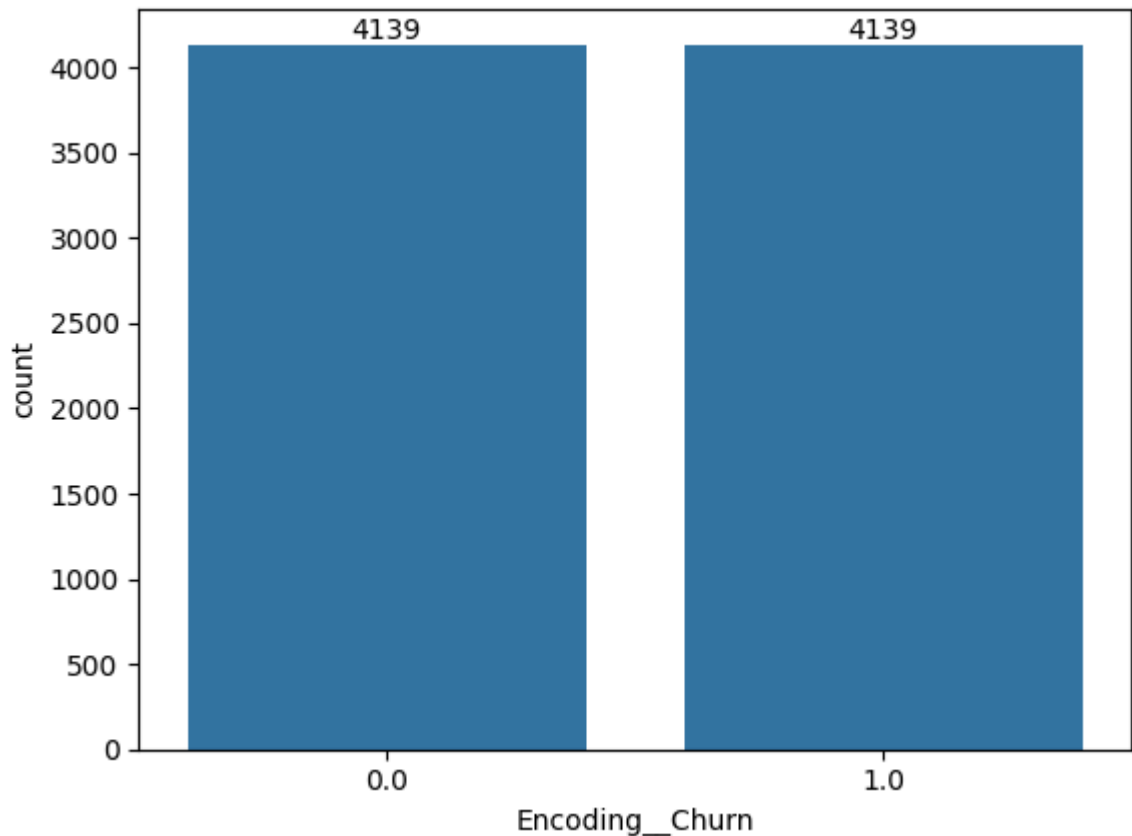
```
In [ ]: plot = sns.countplot(x=y_train['Encoding__Churn'])
        for i in plot.containers:
            plot.bar_label(i)
```



```
In [ ]: SMOTE_Churn = SMOTE()
```

```
In [ ]: x_train, y_train = SMOTE_Churn.fit_resample(x_train,y_train)
```

```
In [ ]: plot = sns.countplot(x=y_train['Encoding__Churn'])
        for i in plot.containers:
            plot.bar_label(i)
```



Model Creation

Random Forest

```
In [110...] RandomSearch_RF = RandomForestClassifier(n_estimators=250,min_samples_split=12,m
```

```
In [110...] start_time = time.time()
RandomSearch_RF.fit(x_train,y_train)
end_time = time.time()
print(f'Time Taken in mins : {(end_time-start_time)/60}')
```

/usr/local/lib/python3.12/dist-packages/sklearn/base.py:1389: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

```
return fit_method(estimator, *args, **kwargs)
```

Time Taken in mins : 0.04739451805750529

```
In [110...] y_train_pred_RF = RandomSearch_RF.predict(x_train)
```

```
In [110...] y_test_pred_RF = RandomSearch_RF.predict(x_test)
```

```
In [110...] y_test_proba_RF = RandomSearch_RF.predict_proba(x_test)[: ,1]
```

```
In [111...] treshold = [round(i,1) for i in np.arange(0.1,1,0.1)]
```

Accuracy

```
In [111...] accuracy_score(y_train,y_train_pred_RF)
```

Out[111...] 0.7949987919787388

```
In [111...] accuracy_score(y_test,y_test_pred_RF)
```

Out[111...] 0.7402413058907026

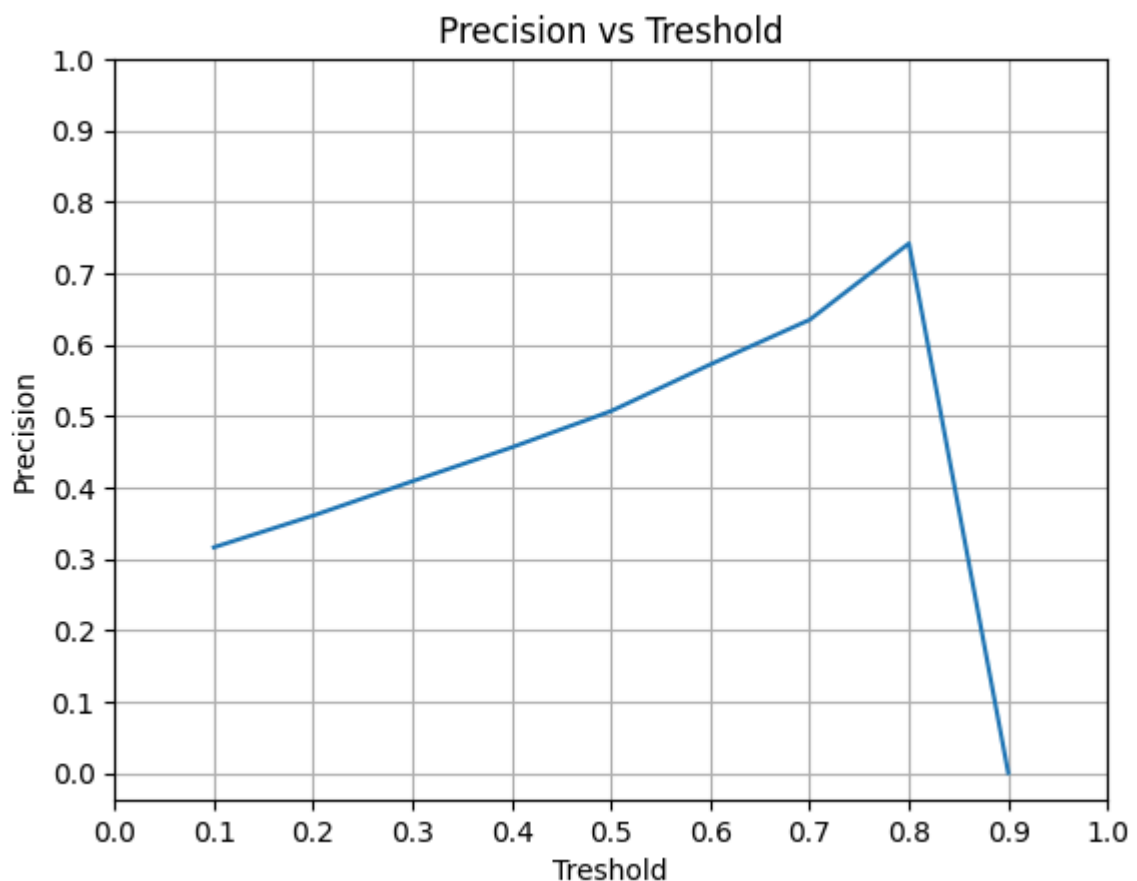
Precision

```
In [111...] precision_score(y_test,y_test_pred_RF)
```

Out[111...] 0.506993006993007

```
In [119...] threshold = [round(i,1) for i in np.arange(0.1,1,0.1)]
precision_test = []
for i in threshold:
    y_pred_test_threshold = (y_test_proba_RF>i).astype('int')
    precision_test.append(precision_score(y_test,y_pred_test_threshold))
plt.plot(threshold, precision_test)
plt.xlabel('Threshold')
plt.xticks([i for i in np.arange(0,1.1,0.1)])
plt.ylabel('Precision')
plt.yticks([i for i in np.arange(0,1.1,0.1)])
plt.title('Precision vs Threshold')
plt.grid()
plt.show()
```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 due to no p
redicted samples. Use `zero\_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))



Recall

```
In [111... recall_score(y_test,y_test_pred_RF)
```

```
Out[111... 0.7754010695187166
```

```
In [120... threshold = [round(i,1) for i in np.arange(0.1,1,0.1)]
recall_test = []
for i in threshold:
    y_pred_test_threshold = (y_test_proba_RF>i).astype('int')
    recall_test.append(recall_score(y_test,y_pred_test_threshold))
plt.plot(threshold, recall_test)
plt.xlabel('Threshold')
plt.xticks([i for i in np.arange(0,1.1,0.1)])
plt.ylabel('Recall')
plt.yticks([i for i in np.arange(0,1.1,0.1)])
plt.title('Recall vs Threshold')
plt.grid()
plt.show()
```

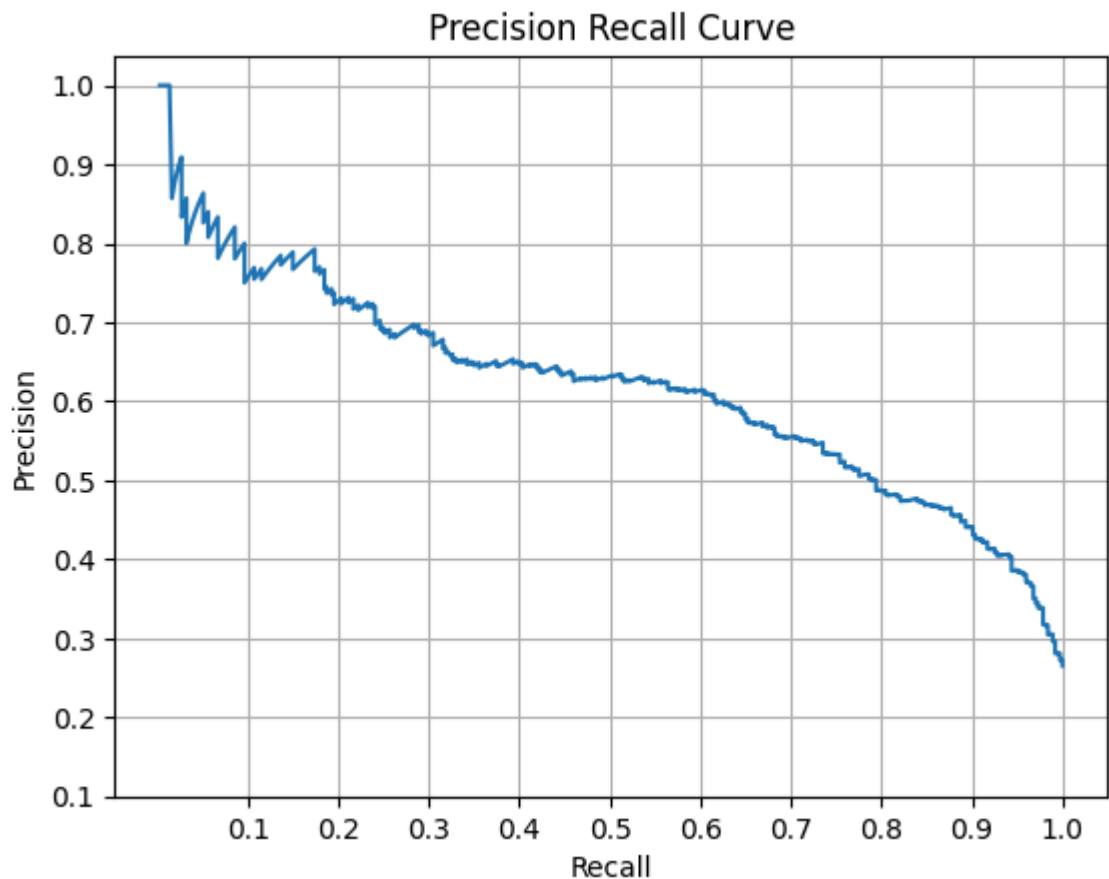


Precision Recall Curve

```
In [111... precision, recall, threshold = precision_recall_curve(y_test,y_test_proba_RF)
```

```
In [112... plt.xlabel('Recall')
plt.ylabel('Precision')
plt.plot(recall[:-1], precision[:-1])
plt.title('Precision Recall Curve')
plt.xticks([i for i in np.arange(0.1,1.1,0.1)])
plt.yticks([i for i in np.arange(0.1,1.1,0.1)])
```

```
plt.grid()
plt.show()
```

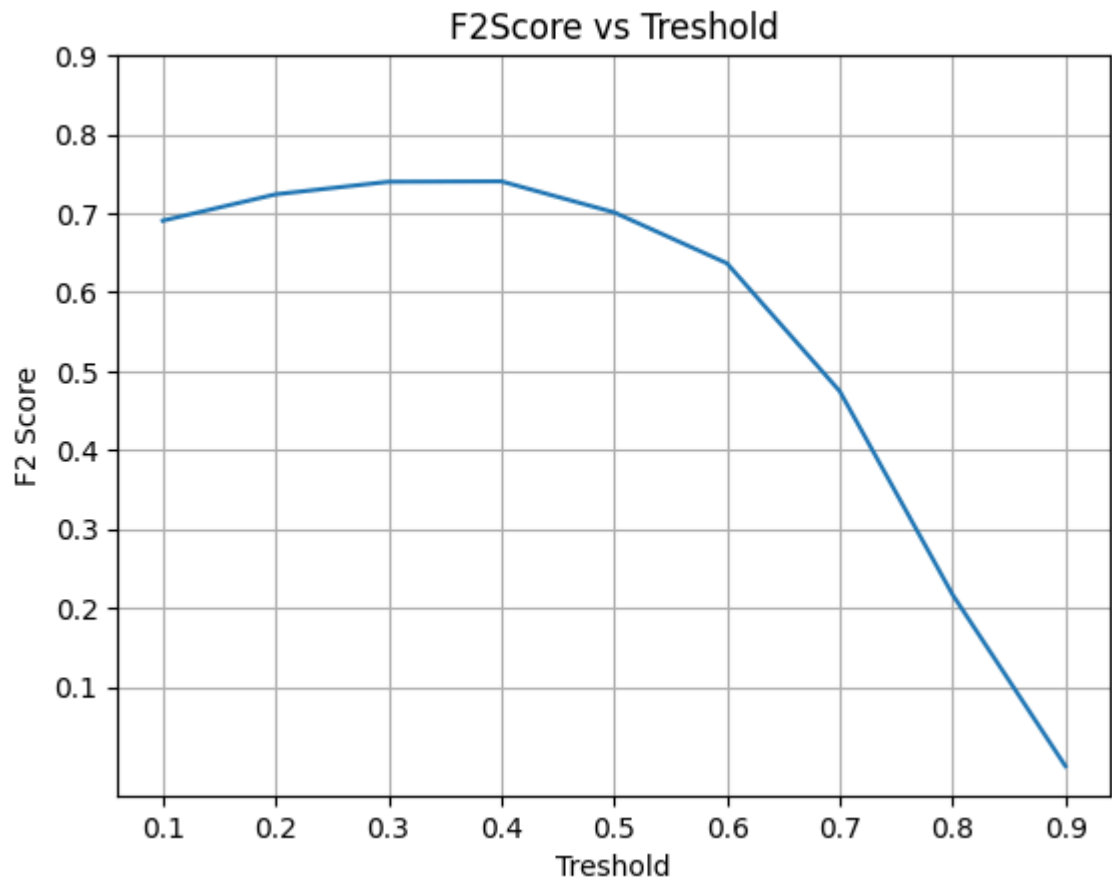


F2 Score

```
In [112...] fbeta_score(y_test,y_test_pred_RF,beta=2)
```

```
Out[112...] 0.7011605415860735
```

```
In [119...] threshold = [round(i,1) for i in np.arange(0.1,1,0.1)]
f2_test = []
for i in threshold:
    y_pred_test_threshold = (y_test_proba_RF>i).astype('int')
    f2_test.append(fbeta_score(y_test,y_pred_test_threshold,beta=2))
plt.plot(threshold, f2_test)
plt.xlabel('Threshold')
plt.xticks([i for i in np.arange(0.1,1,0.1)])
plt.ylabel('F2 Score')
plt.yticks([i for i in np.arange(0.1,1,0.1)])
plt.title('F2Score vs Threshold')
plt.grid()
plt.show()
```

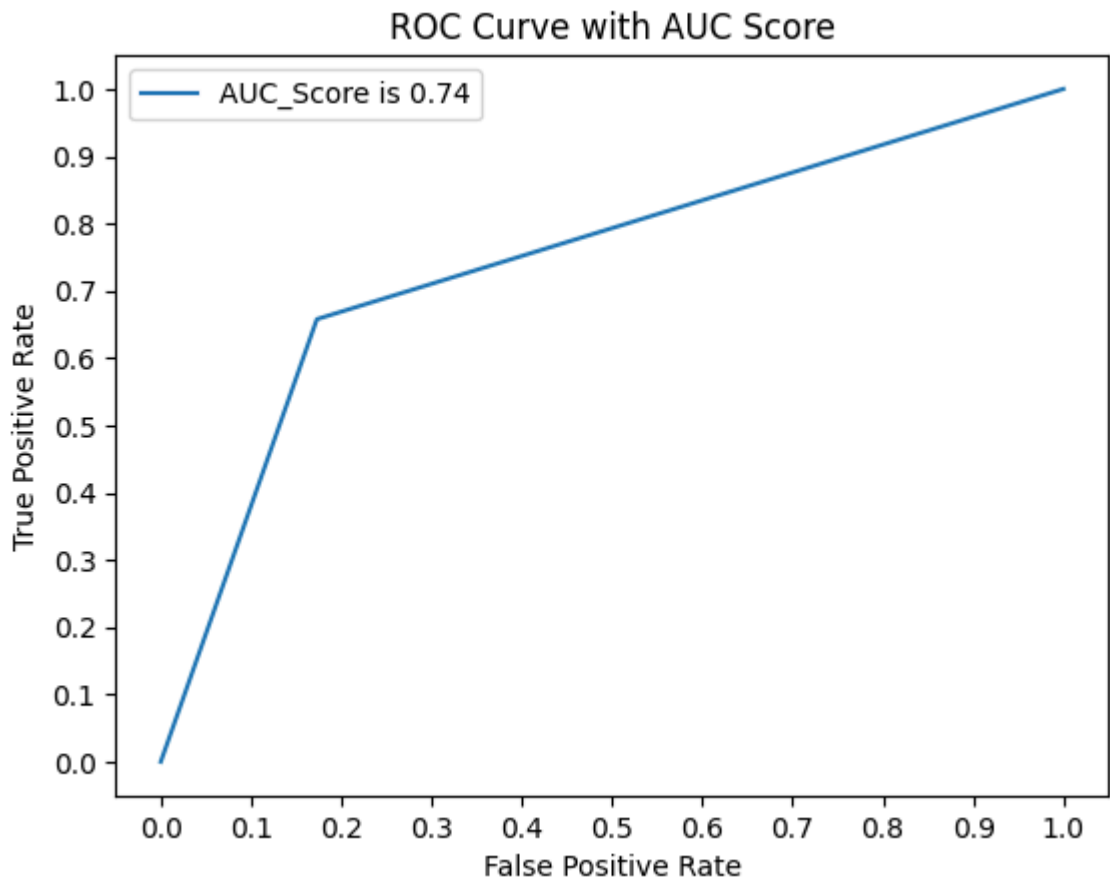


ROC Curve

```
In [112...] fpr, tpr, threshold = roc_curve(y_test,y_test_pred_RF)
```

```
In [112...] auc_score = roc_auc_score(y_test,y_test_pred_RF)
```

```
In [120...] plt.plot(fpr,tpr,label=f'AUC_Score is {round(auc_score,2)}')
plt.xticks([i for i in np.arange(0,1.1,0.1)])
plt.yticks([i for i in np.arange(0,1.1,0.1)])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve with AUC Score')
plt.legend()
plt.show()
```



Gradient Boosting Classifier

```
In [112...] model = GradientBoostingClassifier(n_estimators = 100, min_samples_split=20, min
```

```
In [112...] start_time = time.time()
model.fit(x_train,y_train)
end_time = time.time()
print(f'Time Taken in mins : {(end_time-start_time)/60}')
```

/usr/local/lib/python3.12/dist-packages/sklearn/preprocessing/_label.py:110: Data ConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

y = column_or_1d(y, warn=True)

Time Taken in mins : 0.09034201304117838

```
In [112...] y_train_pred_GBC = RandomSearch_GBC.predict(x_train)
```

```
In [113...] y_test_pred_GBC = RandomSearch_GBC.predict(x_test)
```

```
In [113...] y_test_proba_GBC = RandomSearch_GBC.predict_proba(x_test)[:,:1]
```

```
In [113...] treshold = [round(i,1) for i in np.arange(0.1,1,0.1)]
```

Accuracy

```
In [113...] accuracy_score(y_train,y_train_pred_GBC)
```

```
Out[113...] 0.8840299589272771
```

```
In [113... accuracy_score(y_test,y_test_pred_GBC)
```

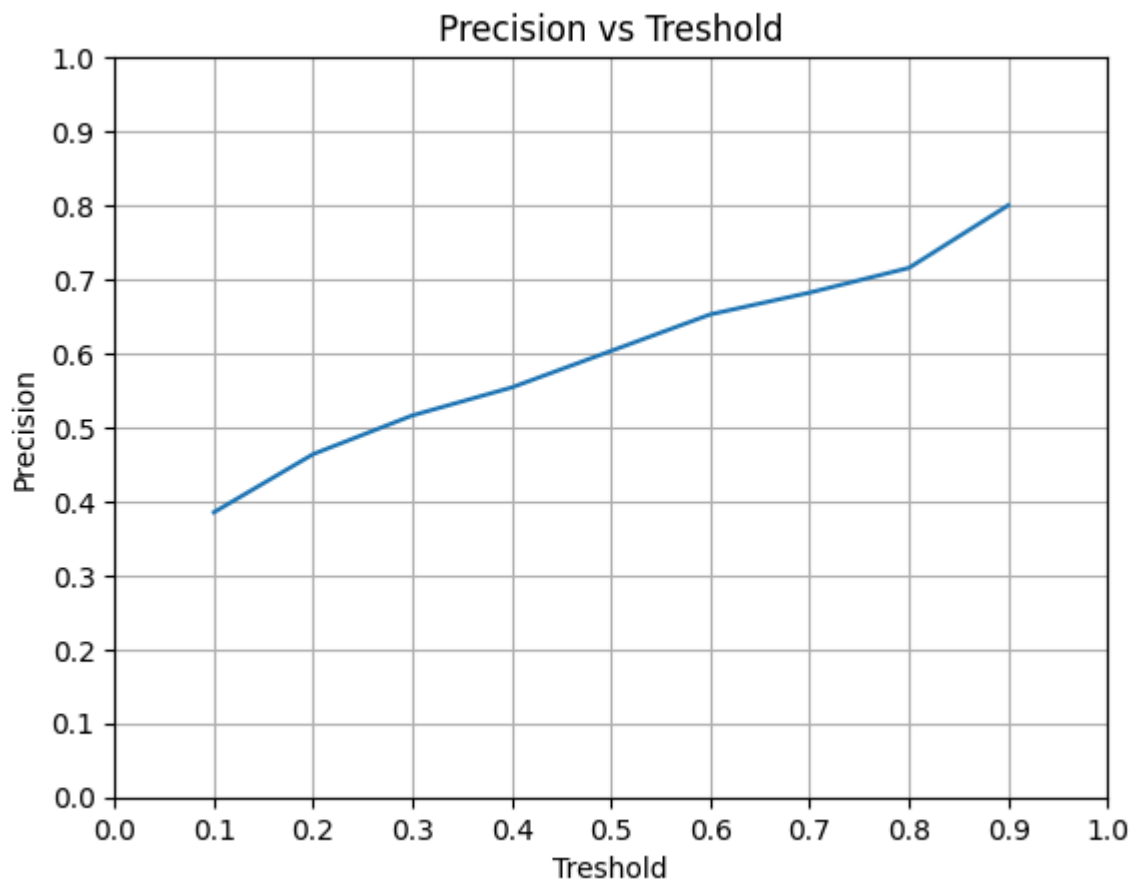
```
Out[113... 0.7892122072391767
```

Precision

```
In [113... precision_score(y_test,y_test_pred_GBC)
```

```
Out[113... 0.6032171581769437
```

```
In [119... threshold = [round(i,1) for i in np.arange(0.1,1,0.1)]
precision_test = []
for i in threshold:
    y_pred_test_threshold = (y_test_proba_GBC>i).astype('int')
    precision_test.append(precision_score(y_test,y_pred_test_threshold))
plt.plot(threshold, precision_test)
plt.xlabel('Threshold')
plt.xticks([i for i in np.arange(0,1.1,0.1)])
plt.ylabel('Precision')
plt.yticks([i for i in np.arange(0,1.1,0.1)])
plt.title('Precision vs Threshold')
plt.grid()
plt.show()
```



Recall

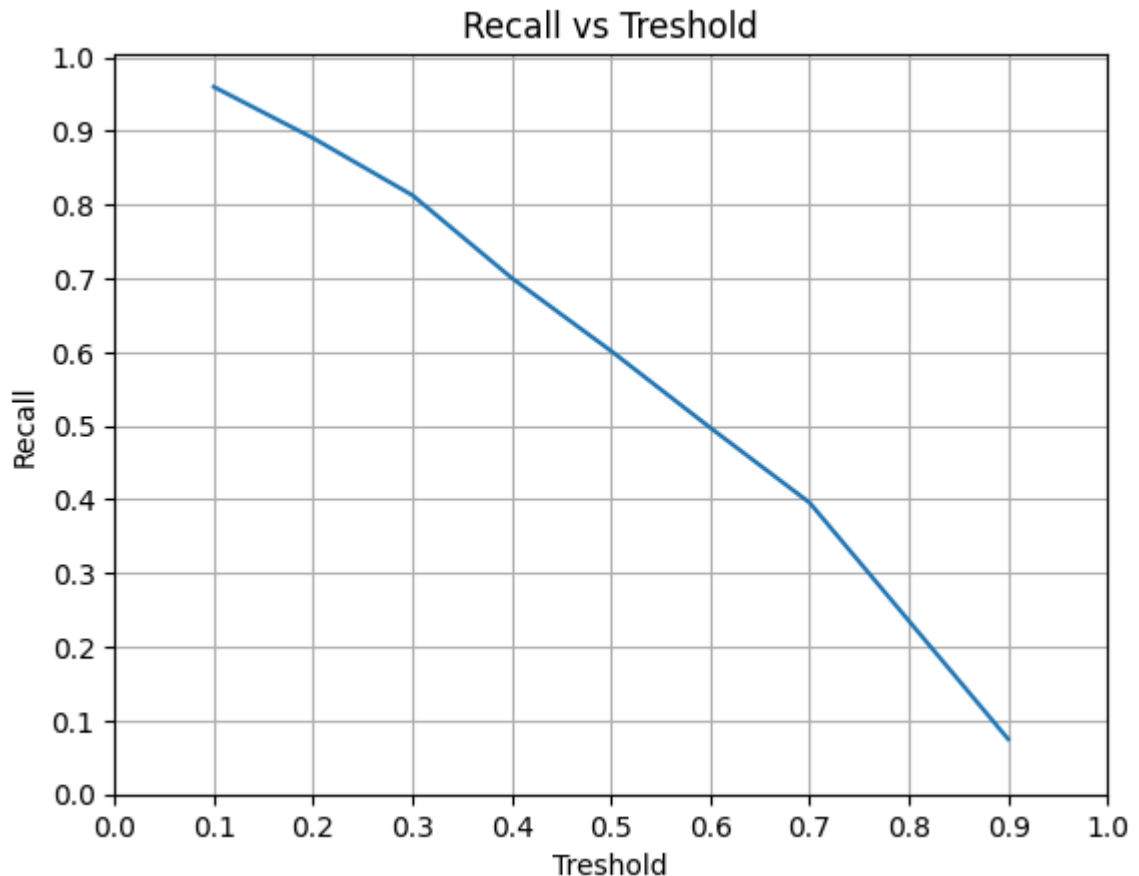
```
In [113... recall_score(y_test,y_test_pred_GBC)
```

```
Out[113... 0.6016042780748663
```

```

In [119... threshold = [round(i,1) for i in np.arange(0.1,1,0.1)]
recall_test = []
for i in threshold:
    y_pred_test_treshold = (y_test_proba_GBC>i).astype('int')
    recall_test.append(recall_score(y_test,y_pred_test_treshold))
plt.plot(threshold, recall_test)
plt.xlabel('Treshold')
plt.xticks([i for i in np.arange(0.0,1.1,0.1)])
plt.ylabel('Recall')
plt.yticks([i for i in np.arange(0.0,1.1,0.1)])
plt.title('Recall vs Treshold')
plt.grid()
plt.show()

```



Precision Recall Curve

```

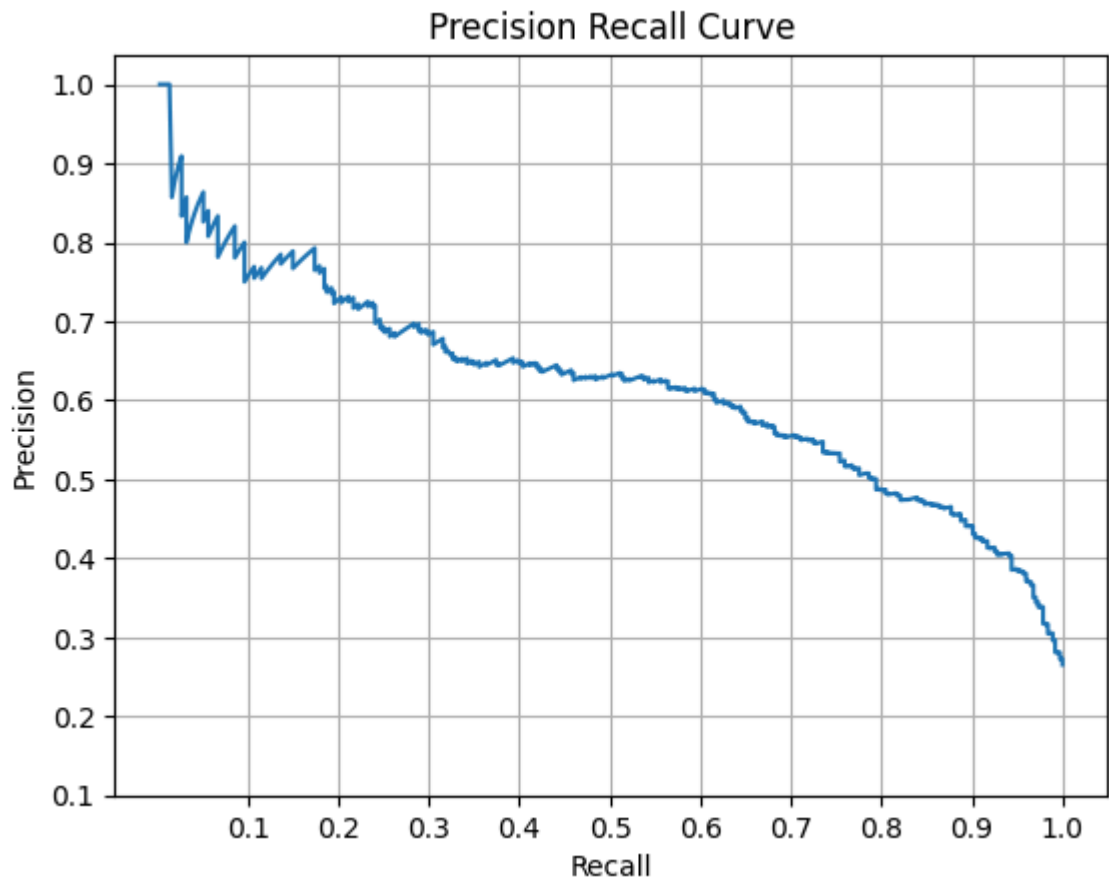
In [114... precision, recall, threshold = precision_recall_curve(y_test,y_test_proba_RF)

```

```

In [114... plt.xlabel('Recall')
plt.ylabel('Precision')
plt.plot(recall[:-1], precision[:-1])
plt.title('Precision Recall Curve')
plt.xticks([i for i in np.arange(0.1,1.1,0.1)])
plt.yticks([i for i in np.arange(0.1,1.1,0.1)])
plt.grid()
plt.show()

```

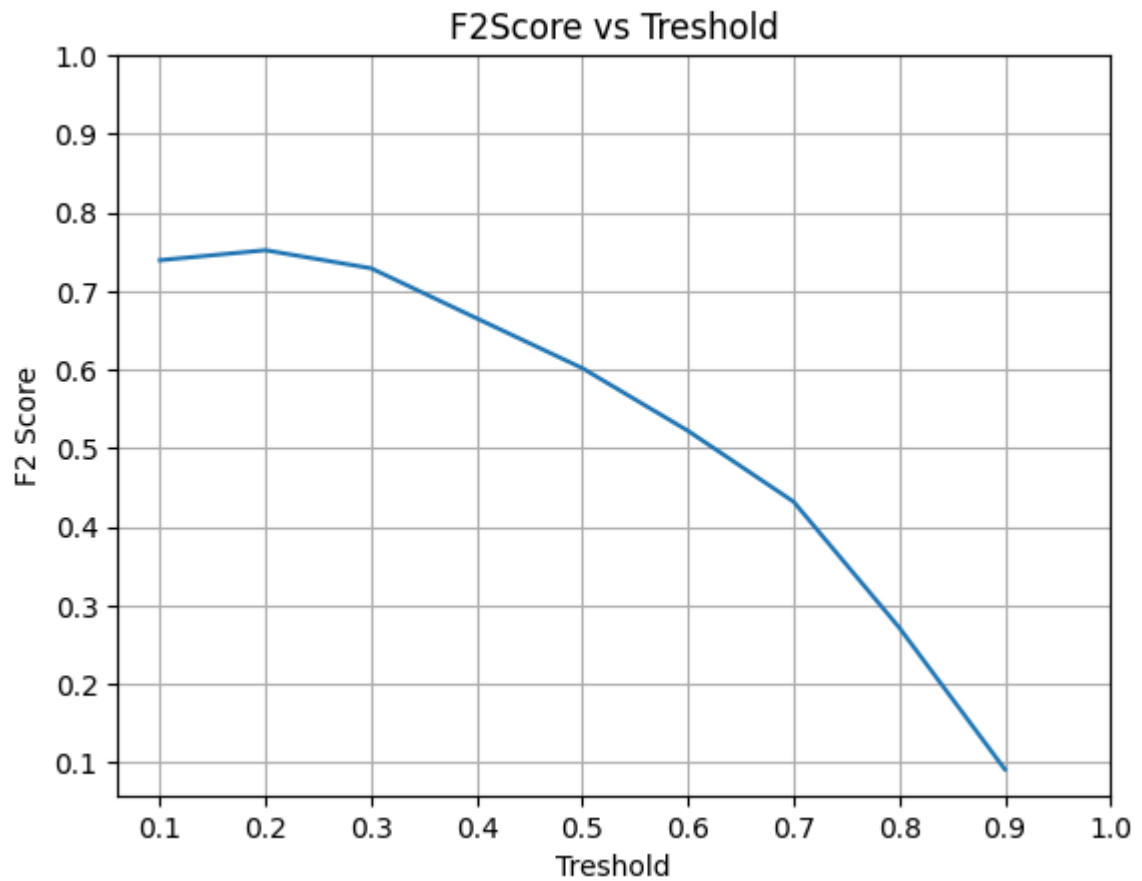


F2 Score

In [114... `fbeta_score(y_test,y_test_pred_GBC,beta=2)`

Out[114... 0.6019261637239165

```
In [119... threshold = [round(i,1) for i in np.arange(0.1,1,0.1)]
f2_test = []
for i in threshold:
    y_pred_test_threshold = (y_test_proba_GBC>i).astype('int')
    f2_test.append(fbeta_score(y_test,y_pred_test_threshold,beta=2))
plt.plot(threshold, f2_test)
plt.xlabel('Threshold')
plt.xticks([i for i in np.arange(0.1,1.1,0.1)])
plt.ylabel('F2 Score')
plt.yticks([i for i in np.arange(0.1,1.1,0.1)])
plt.title('F2Score vs Threshold')
plt.grid()
plt.show()
```

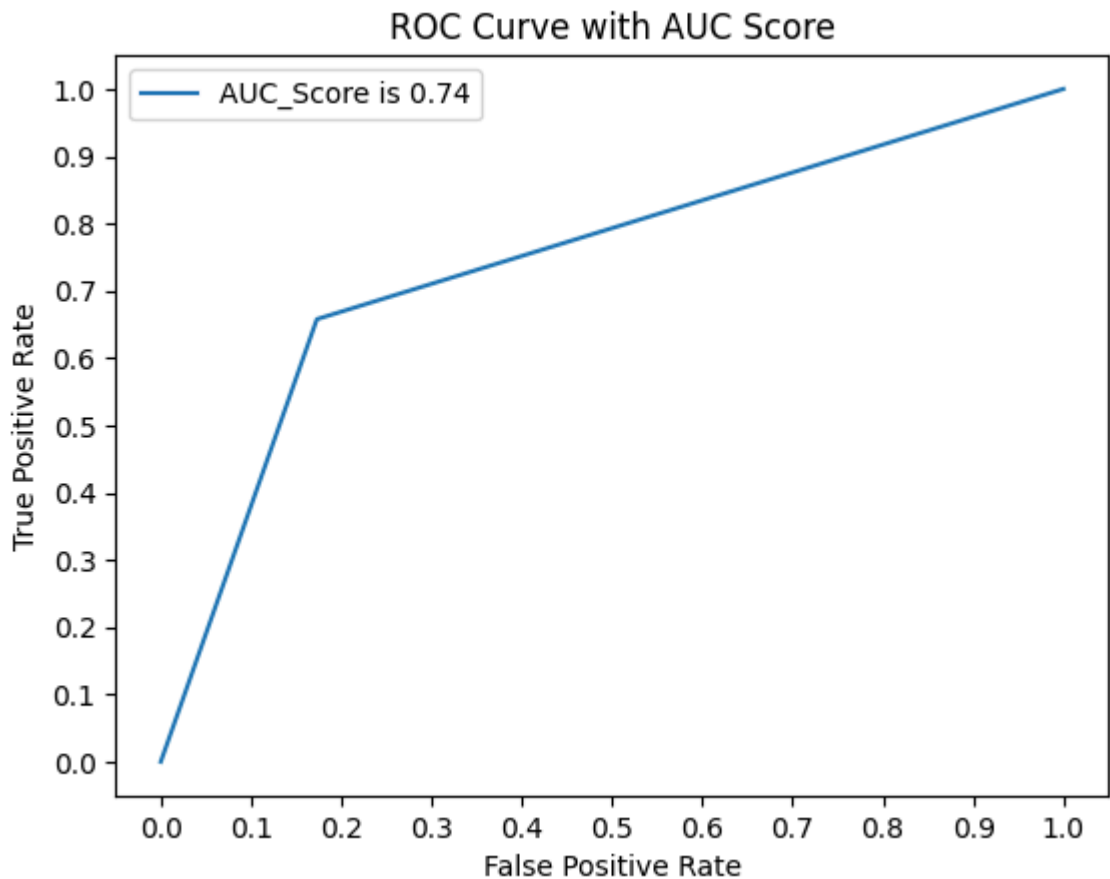


ROC Curve

```
In [114...] fpr, tpr, treshold = roc_curve(y_test,y_test_pred_GBC)
```

```
In [115...] auc_score = roc_auc_score(y_test,y_test_pred_GBC)
```

```
In [120...] plt.plot(fpr,tpr,label=f'AUC_Score is {round(auc_score,2)}')
plt.xticks([i for i in np.arange(0,1.1,0.1)])
plt.yticks([i for i in np.arange(0,1.1,0.1)])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve with AUC Score')
plt.legend()
plt.show()
```

XGBoost Classifier

```
In [115...] model = XGBClassifier(reg_lambda = 2, reg_alpha = 0.5, n_estimators = 50, max_de
```

```
In [115...] start_time = time.time()
model.fit(x_train,y_train)
end_time = time.time()
print(f'Time Taken in mins : {(end_time-start_time)/60}')
```

Time Taken in mins : 0.005554513136545817

```
In [115...] y_train_pred_XGBC = RandomSearch_XGBC.predict(x_train)
```

```
In [115...] y_test_pred_XGBC = RandomSearch_XGBC.predict(x_test)
```

```
In [115...] y_test_proba_XGBC = RandomSearch_XGBC.predict_proba(x_test)[: ,1]
```

```
In [115...] threshold = [round(i,1) for i in np.arange(0.1,1,0.1)]
```

Accuracy

```
In [115...] accuracy_score(y_train,y_train_pred_XGBC)
```

Out[115...] 0.8618023677216718

```
In [115...] accuracy_score(y_test,y_test_pred_XGBC)
```

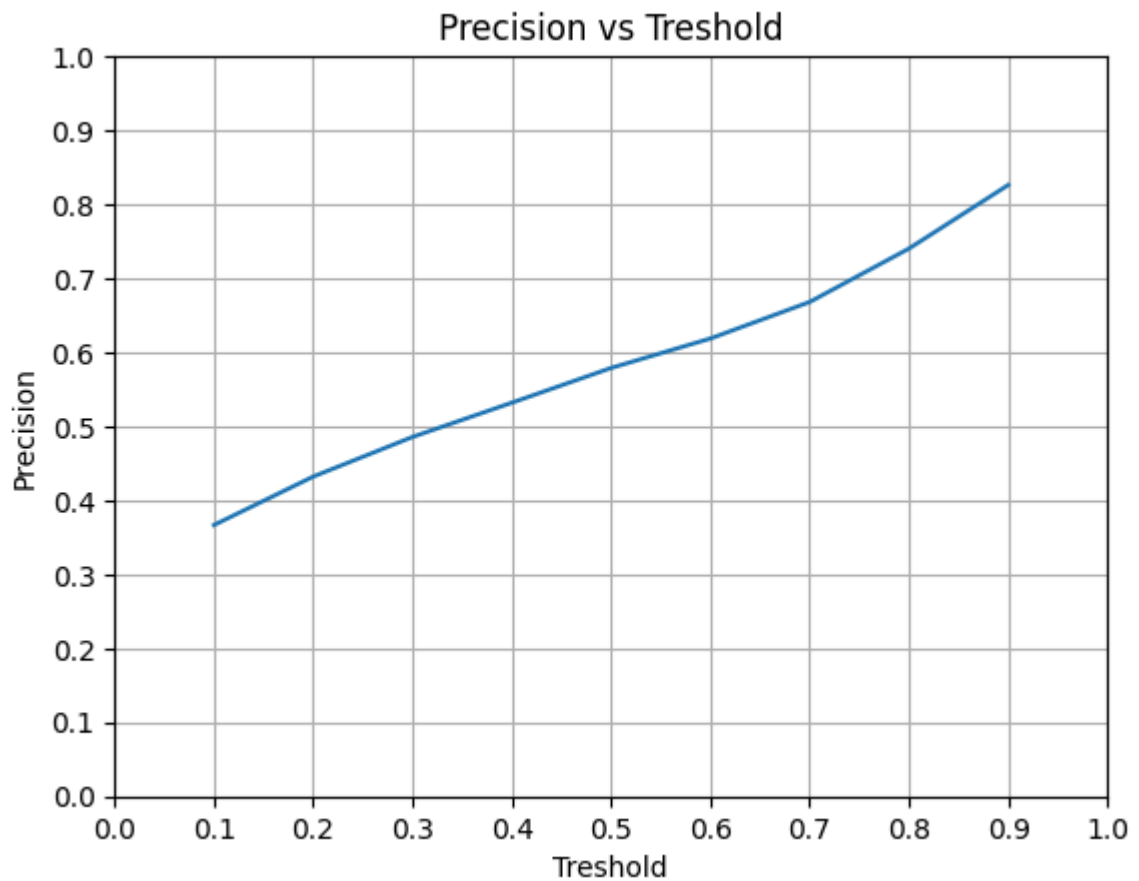
Out[115...] 0.7821149751596878

Precision

```
In [116... precision_score(y_test,y_test_pred_XGBC)
```

```
Out[116... 0.5788235294117647
```

```
In [118... threshold = [round(i,1) for i in np.arange(0.1,1,0.1)]
precision_test = []
for i in threshold:
    y_pred_test_threshold = (y_test_proba_XGBC>i).astype('int')
    precision_test.append(precision_score(y_test,y_pred_test_threshold))
plt.plot(threshold, precision_test)
plt.xlabel('Threshold')
plt.xticks([i for i in np.arange(0,1.1,0.1)])
plt.ylabel('Precision')
plt.yticks([i for i in np.arange(0,1.1,0.1)])
plt.title('Precision vs Threshold')
plt.grid()
plt.show()
```



Recall

```
In [116... recall_score(y_test,y_test_pred_XGBC)
```

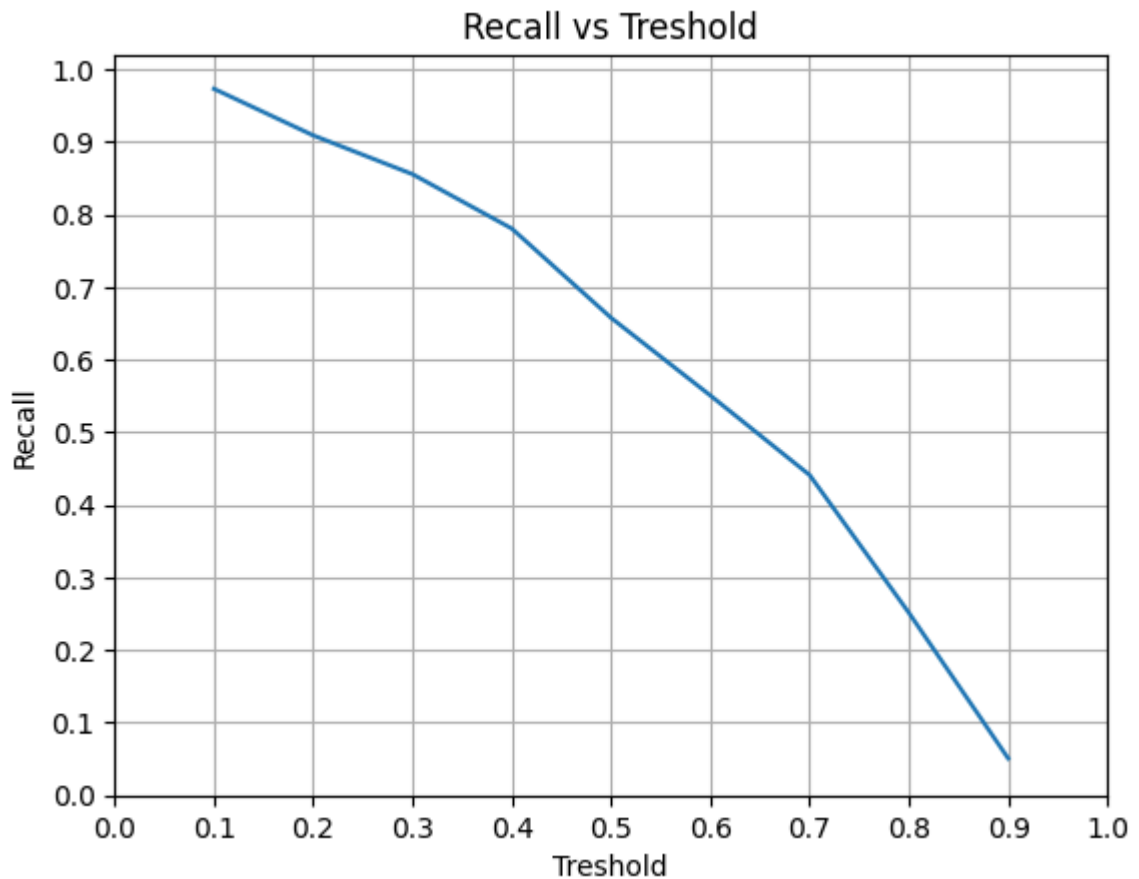
```
Out[116... 0.6577540106951871
```

```
In [118... threshold = [round(i,1) for i in np.arange(0.1,1,0.1)]
recall_test = []
for i in threshold:
    y_pred_test_threshold = (y_test_proba_XGBC>i).astype('int')
```

```

recall_test.append(recall_score(y_test,y_pred_test_treshold))
plt.plot(treshold, recall_test)
plt.xlabel('Treshold')
plt.xticks([i for i in np.arange(0.0,1.1,0.1)])
plt.ylabel('Recall')
plt.yticks([i for i in np.arange(0.0,1.1,0.1)])
plt.title('Recall vs Treshold')
plt.grid()
plt.show()

```



Precision Recall Curve

```

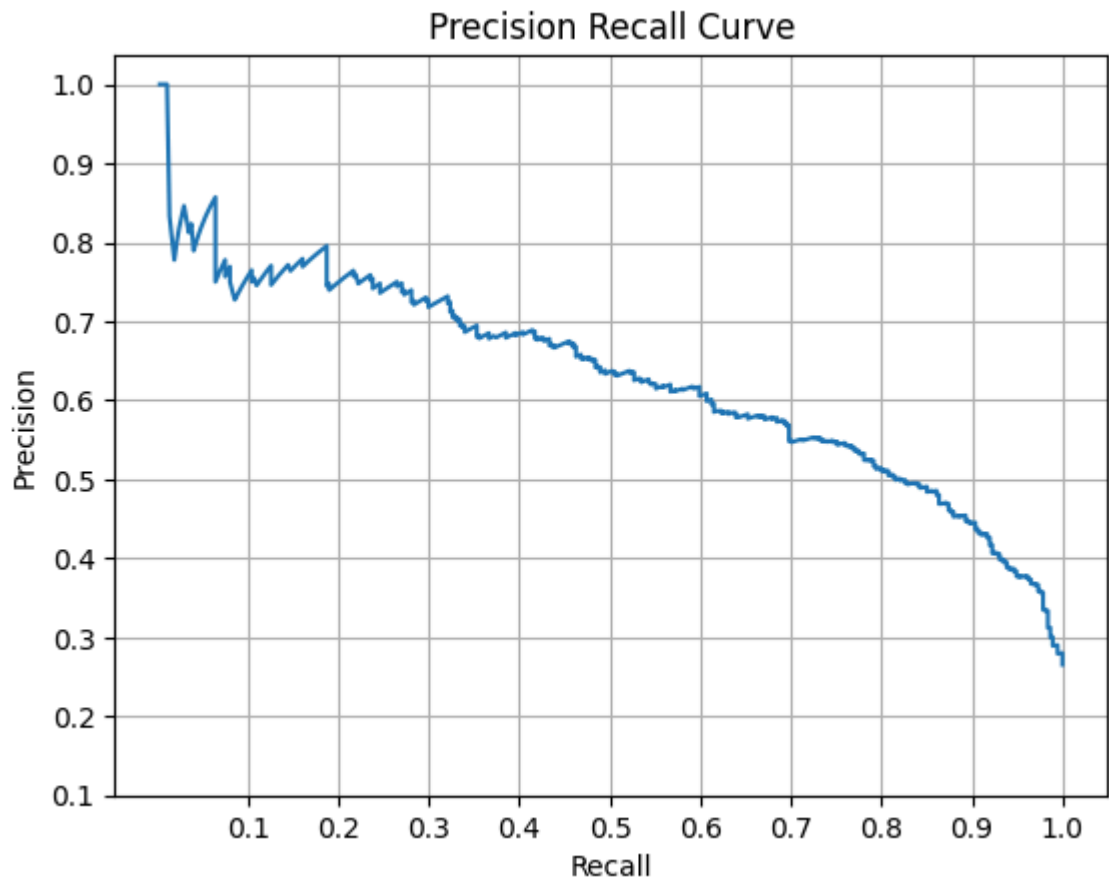
In [116...] precision, recall, threshold = precision_recall_curve(y_test,y_test_proba_XGBC)

```

```

In [116...] plt.xlabel('Recall')
plt.ylabel('Precision')
plt.plot(recall[:-1], precision[:-1])
plt.title('Precision Recall Curve')
plt.xticks([i for i in np.arange(0.1,1.1,0.1)])
plt.yticks([i for i in np.arange(0.1,1.1,0.1)])
plt.grid()
plt.show()

```

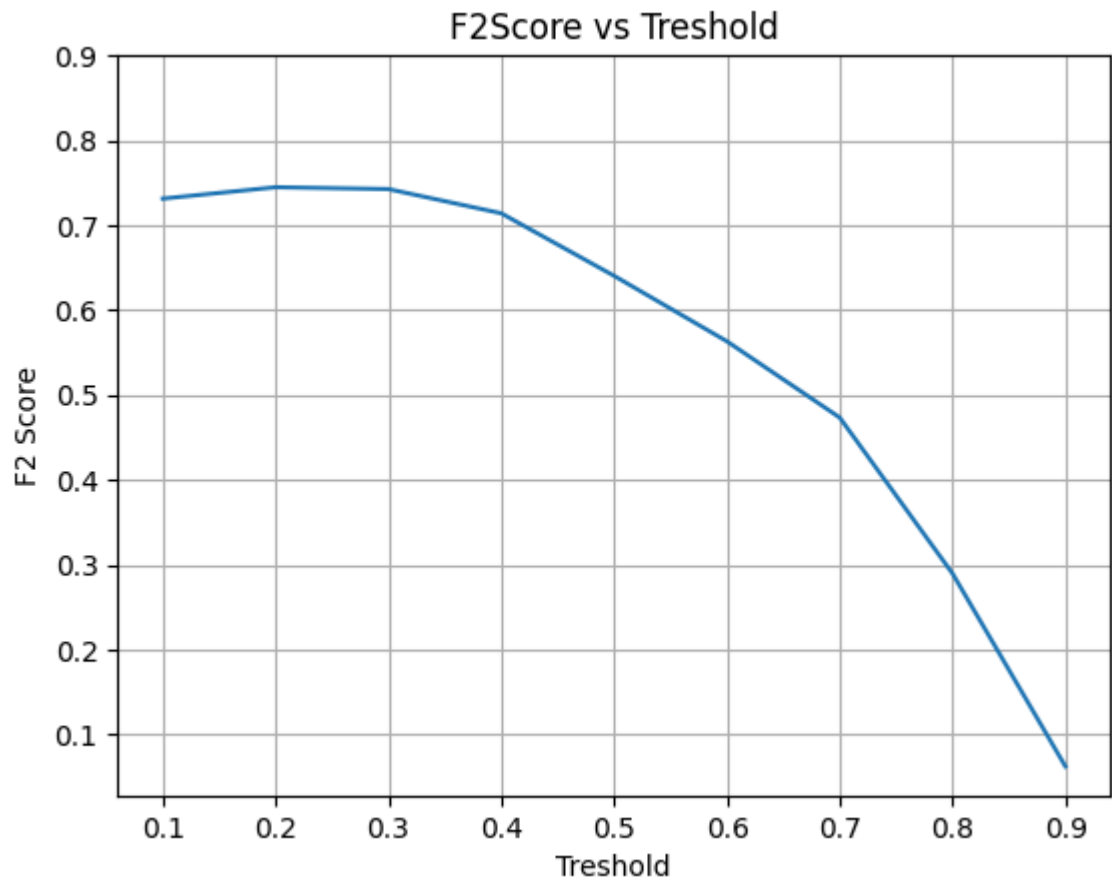


F2 Score

In [116... `fbeta_score(y_test,y_test_pred_XGBC,beta=2)`

Out[116... 0.6402915148360229

```
In [118... threshold = [round(i,1) for i in np.arange(0.1,1,0.1)]
f2_test = []
for i in threshold:
    y_pred_test_threshold = (y_test_proba_XGBC>i).astype('int')
    f2_test.append(fbeta_score(y_test,y_pred_test_threshold,beta=2))
plt.plot(threshold, f2_test)
plt.xlabel('Threshold')
plt.xticks([i for i in np.arange(0.1,1,0.1)])
plt.ylabel('F2 Score')
plt.yticks([i for i in np.arange(0.1,1,0.1)])
plt.title('F2Score vs Threshold')
plt.grid()
plt.show()
```

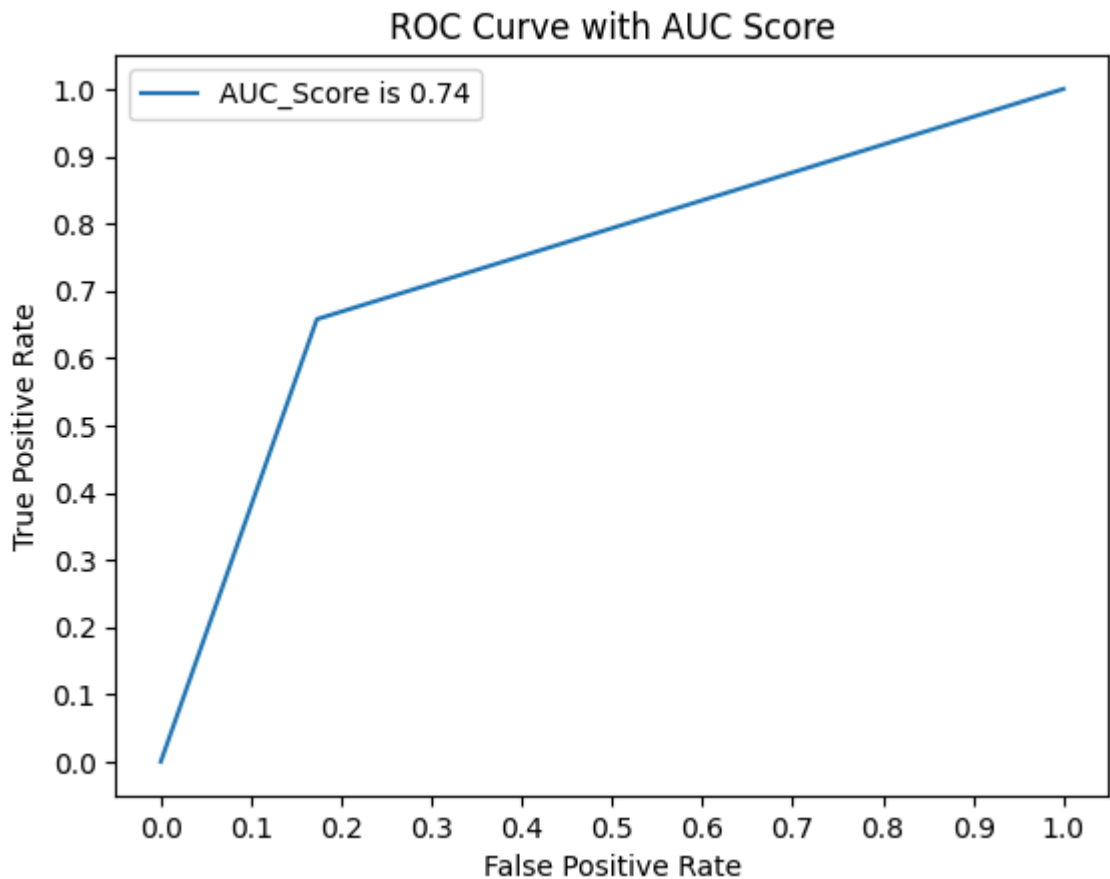


ROC Curve

```
In [117...] fpr, tpr, treshold = roc_curve(y_test,y_test_pred_XGBC)
```

```
In [117...] auc_score = roc_auc_score(y_test,y_test_pred_XGBC)
```

```
In [120...] plt.plot(fpr,tpr,label=f'AUC_Score is {round(auc_score,2)}')
plt.xticks([i for i in np.arange(0,1.1,0.1)])
plt.yticks([i for i in np.arange(0,1.1,0.1)])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve with AUC Score')
plt.legend()
plt.show()
```



Comparing the models

```
In [117...] print(f'Recall of Random Forest is {recall_score(y_test,y_test_pred_RF)}')
print(f'Recall of Gradient Boosting Classifier is {recall_score(y_test,y_test_pred_XGBC)}')
print(f'Recall of XGBoost Classifier is {recall_score(y_test,y_test_pred_XGBC)}')
```

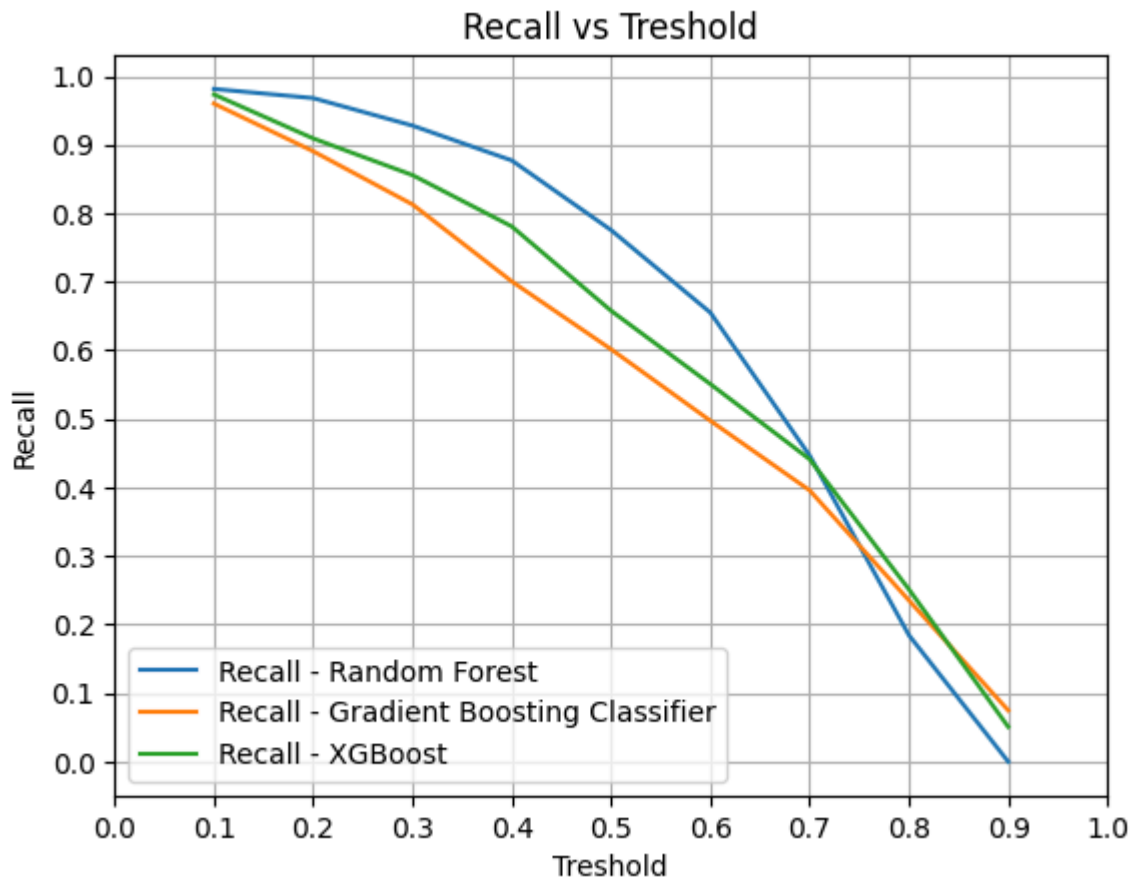
Recall of Random Forest is 0.7754010695187166
 Recall of Gradient Boostin Classifier is 0.6016042780748663
 Recall of XGBoost Classifier is 0.6577540106951871

```
In [117...] print(f'Recall of Random Forest is {fbeta_score(y_test,y_test_pred_RF,beta=2)}')
print(f'Recall of Gradient Boosting Classifier is {fbeta_score(y_test,y_test_pred_XGBC,beta=2)}')
print(f'Recall of XGBoost Classifier is {fbeta_score(y_test,y_test_pred_XGBC,beta=2)}')
```

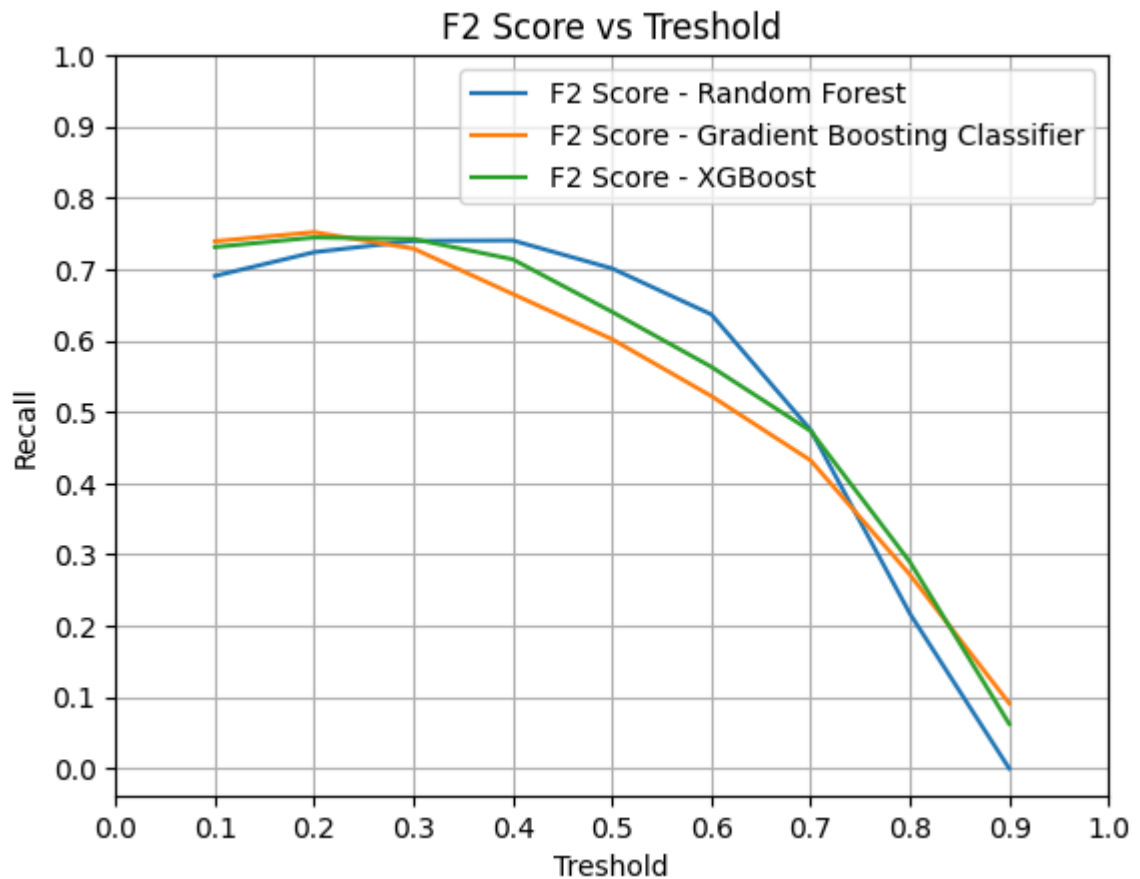
Recall of Random Forest is 0.7011605415860735
 Recall of Gradient Boosting Classifier is 0.6019261637239165
 Recall of XGBoost Classifier is 0.6402915148360229

```
In [118...] threshold = [round(i,1) for i in np.arange(0.1,1,0.1)]
recall_RF = []
recall_GBC = []
recall_XGBC = []
for i in threshold:
    y_pred_test_threshold_RF = (y_test_proba_RF>i).astype('int')
    y_pred_test_threshold_GBC = (y_test_proba_GBC>i).astype('int')
    y_pred_test_threshold_XGBC = (y_test_proba_XGBC>i).astype('int')
    recall_RF.append(recall_score(y_test,y_pred_test_threshold_RF))
    recall_GBC.append(recall_score(y_test,y_pred_test_threshold_GBC))
    recall_XGBC.append(recall_score(y_test,y_pred_test_threshold_XGBC))
plt.plot(threshold, recall_RF,label= 'Recall - Random Forest')
plt.plot(threshold, recall_GBC,label= 'Recall - Gradient Boosting Classifier')
```

```
plt.plot(threshold, recall_XGBC, label= 'Recall - XGBoost')
plt.xlabel('Threshold')
plt.xticks([i for i in np.arange(0.0,1.1,0.1)])
plt.ylabel('Recall')
plt.yticks([i for i in np.arange(0.0,1.1,0.1)])
plt.title('Recall vs Threshold')
plt.legend()
plt.grid()
plt.show()
```



```
In [119... threshold = [round(i,1) for i in np.arange(0.1,1,0.1)]
f2_RF = []
f2_GBC = []
f2_XGBC = []
for i in threshold:
    y_pred_test_threshold_RF = (y_test_proba_RF>i).astype('int')
    y_pred_test_threshold_GBC = (y_test_proba_GBC>i).astype('int')
    y_pred_test_threshold_XGBC = (y_test_proba_XGBC>i).astype('int')
    f2_RF.append(fbeta_score(y_test,y_pred_test_threshold_RF,beta=2))
    f2_GBC.append(fbeta_score(y_test,y_pred_test_threshold_GBC,beta=2))
    f2_XGBC.append(fbeta_score(y_test,y_pred_test_threshold_XGBC,beta=2))
plt.plot(threshold, f2_RF, label= 'F2 Score - Random Forest')
plt.plot(threshold, f2_GBC, label= 'F2 Score - Gradient Boosting Classifier')
plt.plot(threshold, f2_XGBC, label= 'F2 Score - XGBoost')
plt.xlabel('Threshold')
plt.xticks([i for i in np.arange(0.0,1.1,0.1)])
plt.ylabel('Recall')
plt.yticks([i for i in np.arange(0.0,1.1,0.1)])
plt.title('F2 Score vs Threshold')
plt.legend()
plt.grid()
plt.show()
```



```
In [119... print(f'AUC Score of Random Forest is {roc_auc_score(y_test,y_test_pred_RF)}')
print(f'AUC Score of Random Forest is {roc_auc_score(y_test,y_test_pred_GBC)}')
print(f'AUC Score of Random Forest is {roc_auc_score(y_test,y_test_pred_XGBC)}')
```

```
AUC Score of Random Forest is 0.7514686507013872
AUC Score of Random Forest is 0.7293045544963703
AUC Score of Random Forest is 0.7424035753959027
```

In this case recall is more important than precision so precision, f2 score is compared among the models. Random Forest Classifier at 0.4 treshold highest recall and f2score.

On comparing AUC score Random Forest model posses high AUC score

```
In [120... y_pred_final = (y_test_proba_RF>=0.4).astype('int')
print(f'Recall score of Random Forest model at threshold 0.4 - {recall_score(y_t
print(f'F2 score of Random Forest model at threshold 0.4 - {fbeta_score(y_test,y
```

```
Recall score of Random Forest model at threshold 0.4 - 0.8770053475935828
F2 score of Random Forest model at threshold 0.4 - 0.7404063205417607
```